

Project continuation status

04/29/2026 00:04:41

Do you remember where we last left off on this project

Tool: Looking for relevant chats in this project... You're working on Aurua, your Track A project for the Agentic Technologies...

File: Phase_2_AI_Chat_Discussion.pdf This is a pdf of our previous chat conversation. We finished phase 2

Got it — thanks for sharing the full transcript. I'm caught up on everything: the Phase 2 submission went out with the Phase ...

File: Jorge Urias Phase 2 Feedback.docx File: As a reminder here's the project requirements. We also received feedback on our...

I haven't actually seen the Phase 2 feedback yet — your message references it ("we also received feedback on our submissio...

File: Jorge Urias Phase 2 Feedback.docx Here's the feedback

I still can't see the contents of the feedback doc — only the file path is showing up, not the actual text. The .docx isn't...

Well I'm already past the due date technically. It was due Sunday and its Tuesday night. We have to work on Phase 3 and try t...

Got it. Phase 3 is overdue, presentation is Thursday, and the goal tonight is a finished, demoable project. Let me give you the...

Q: On the orchestration justification — keep custom or migrate to LangGraph? A: Keep custom, write justification (faster, ~30...

Good. Let me push back on one of these before we start, because the time math matters. You picked full video with TTS +...

Q: Tiered build (Tier 1 → 2 → 3) or full send on the audio pipeline? A: Tiered approach — ship whichever tier I reach Q: Do you...

Good. Tiered approach with OpenAI TTS as the audio path is the smart call — you already have OpenAI set up from Phase 2,...

Q: Where is the aurua project right now — do you have it open? A: Yes, on this machine — I'll paste files/run commands Q:...

Good. Let me kick this off properly. Step 1 — Install Manim (run this now in parallel) Manim has system dependencies that ta...

File: Here's my project directory tree

Good. I have a clear picture of your project state. A few things I notice: Architecture diagram is a PDF, not editable — we'll ne...

File:

Perfect. Now I have what I need. Two things to flag before writing patches: Constant name correction. Your config.py uses...

remind me why we are changing config.py and intent_grounding.py?

Good question to stop and ask — make sure you understand why before applying changes. Both changes address the F1 / F2...

File: state.json File: 00_query_expansion.txt File: 01_intent.txt File: 02_planner_initial.txt File: 02_planner_retry_0.txt File:...

Important data. Let me read this carefully — because the headline ("verification_exhausted, same as before") hides what...

File: state.json File: 00_query_expansion.txt File: 01_intent.txt File: 02_planner_initial.txt File: 02_planner_retry_0.txt File:...

Wait — read your command line carefully. You ran TC1 against 3b1b_ch4.txt, not 3b1b_ch3.txt. --source...

Q: What's the move? A: Re-run TC1 against the correct source (Ch3) — likely passes, becomes the demo

Good. Run this now while I keep building: rm -rf data/index/ python -m src.run \ --question "What does the gradient of the co...

File: state.json File: File: 00_query_expansion.txt File: 01_intent.txt File: 02_planner_initial.txt File: 03_verifier_attempt_0.txt File:...

Excellent. TC1 verified first-try, 4 calls, \$0.07, 30 seconds, all 4 scenes grounded with confidence 0.85-0.95. This is the canonic...

File: state.json File: File: 00_query_expansion.txt File: 01_intent.txt File: 02_planner_initial.txt File: 02_planner_retry_0.txt File:...

Two big wins, one specific bug to fix. Let me unpack: Wins: Manim works. The test rendered cleanly through 6 animations. Th...

File: state.json File: File: concat_list.txt File: scene_01.py File: scene_02.py File: scene_03.py File: scene_04.py File:...

End-to-end pipeline working. Take the win. Status: rendered. 4 verified scenes. 4 Manim renders. ffmpeg concat to a single...

File: Jorge Urias Phase 2 Feedback.pdf File: Great! Before going into the audio, do we need to do any more runs for the full...

Smart move to pause and audit before sprinting further. Let me walk through what you have and what you genuinely still...

Q: Given the gaps I just listed, which path tonight? A: Hybrid — doc gaps + missing runs first, audio if time permits at end

Good. Let's move. Step 1 — Re-run TC5 with v0.3 The FAISS index is currently built from Ch3 (last run was TC1). TC5 is on Ch4....

Q: Should TC5 v0.3 run with rendering or without? A: Run full pipeline — burn the \$0.50, get a second rendered video as bonus

Good. Run TC5 with the full pipeline: rm -rf data/index/ python -m src.run \ --question "Why does the chain rule give us a sum...

File: state.json File: File: 00_query_expansion.txt File: 01_intent.txt File: 02_planner_initial.txt File: 02_planner_retry_0.txt File:...

Read carefully. This is exactly the F4 finding repeating itself on TC5 — and the verifier surfaced the real root cause in plain...

File: state.json Output: python scripts/run_tc6_baseline.py \ — --question "Why is the derivative of the cost with respect to a...

TC6 run-2 reproduces the baseline finding cleanly. Let me make this concrete: TC6 reproducibility check Run 1 (tc6_ddbcd2db...
Q: Final report format — what works best for you? A: Final report as Markdown + I render to PDF in the sandbox
Good. Writing the final report now. Given how long this is, I'll write it as a single large file in the sandbox, then render to PDF,...

 Do you remember where we last left off on this project

4/28/2026, 8:41:04 PM

Tool: Looking for relevant chats in this project...

You're working on **Aurua**, your Track A project for the Agentic Technologies course. Last we left off, we had just hit the Ch3 transcript problem and I'd handed you a cleaned version of `3b1b_ch3.txt` plus updated `clean_transcript.py` and `apply_fixes.py` scripts.

Here's the state of play before that interruption:

What's done:

- Phase 2 architecture doc, role definitions, coordination logic, tools/memory design, risk plan — all written
- Agents 1–3 built and running; TC4 executed successfully end-to-end
- TC6 baseline (single-prompt, no retrieval/verifier) ran — produced 4 scenes from Ch4

What's still open:

- **Real evaluation data** — biggest gap. You've got 5 test cases in `test_cases.csv` but only TC4 fully run. TC5 and TC6 were the next ones queued up.
- **\$5.4 of the Phase 2 doc** — still has the fabricated 30% figure. Needs to either come out or get backed by real ablation numbers.
- **Submission packet** — PDF not assembled, GitHub repo not created, screenshot index and file list not done.
- **Architecture diagram** — drafted inline in the MD doc, still needs export to standalone PDF.

The natural next step when you got pulled away was: drop the cleaned Ch3 file into `data/transcripts/`, then run TC5 against Ch4 (the chain-rule-over-paths question) to generate a second real evaluation trace. That gives you two grounded runs to write up instead of one.

Want to pick up there, or has something shifted since?

4/28/2026, 8:41:17 PM



File: Phase_2_AI_Chat_Discussion.pdf

This is a pdf of our previous chat conversation. We finished phase 2

4/28/2026, 8:42:26 PM

Got it — thanks for sharing the full transcript. I'm caught up on everything: the Phase 2 submission went out with the Phase 2 doc PDF + GitHub repo, and you correctly pushed back when I'd over-scoped the submission packet and screenshot index into Phase 2 work that actually belongs to Phase 3.

So now you're sitting at the start of Phase 3 with a clean inheritance from Phase 2:

What's already drafted and waiting for you:

- `docs/submission_packet.pdf` (with three placeholders to fill in: submission date, repo URL, re-render command)
- `docs/screenshots/screenshot_index.md` (7 screenshots specified, captures still needed)
- `docs/architecture_diagram.pdf`

What `version_notes.md` already wrote as your Phase 3 priority list:

1. Fix retrieval coverage (F1/F2) — chunk size ↓, top-k ↑, possibly query expansion
2. Add a larger-corpus stress test to justify the architecture in its intended regime (addresses F3)
3. Wire the rendering branch (Agents 4a Animation Coder, 4b Narration Writer, deterministic Renderer)
4. Generate at least one actual video end-to-end

The narrative arc you set up for Phase 3: The before/after story. TC2 and TC5 exhausted the verifier in Phase 2 because of the retrieval coverage gap. If you fix the retrieval config and re-run those exact test cases, the "before" trace (already archived in `eval/runs_saved/phase2_evaluation/`) and the "after" trace become your strongest demo evidence. The rubric specifically rewards this kind of "what changed after testing" narrative.

A few things worth deciding before we dive back in:

1. **Did Phase 2 grading come back yet?** If it did, the feedback likely shifts what we prioritize.
2. **What's the Phase 3 deadline and your rough time budget?** That determines whether to fully wire the rendering branch or scope down to "one rendered scene as proof of concept."
3. **Where do you want to start?** The retrieval fix is the cheapest, highest-leverage move — re-run TC2 with smaller chunks and watch it pass. The rendering branch is where Phase 3 projects most often collapse, so starting there carries more risk.

Where do you want to pick up?

4/28/2026, 8:42:45 PM



File: *Jorge Urias Phase 2 Feedback.docx*

File:

As a reminder here's the project requirements. We also received feedback on our submission

4/28/2026, 8:44:38 PM

I haven't actually seen the Phase 2 feedback yet — your message references it ("we also received feedback on our submission") but the file that came through is the project requirements/spec doc, not the feedback. The `Jorge_Urias_Phase_2_Feedback.docx` is referenced in your uploads but its contents aren't visible to me.

Can you do one of the following:

1. **Re-upload the feedback doc** — sometimes .docx files don't extract cleanly on the first try
2. **Paste the feedback as text** directly into a message
3. **Tell me the gist** if it's short

Once I can see what the grader actually said, I can map it against where you currently stand and figure out what shifts in priority for Phase 3. Without the feedback, I'd be guessing — and given the project's whole emphasis on honest evidence-based work, guessing at what the grader said is exactly the wrong move.

The requirements doc is useful to have in context though. One thing I notice re-reading it: the Phase 3 rubric weights "Failure analysis and iteration" at 15 points and specifically asks for "what changed after testing." Your Phase 2 left you with three documented failures (F1/F2 retrieval coverage, F3 architecture-vs-baseline) and a clear iteration plan in `version_notes.md`. That before/after arc is worth 15 of the 25 percentage points Phase 3 contributes to your final grade — it's the highest-leverage thing in the rubric for your specific situation.

But let's see the feedback first before deciding what to do.

4/28/2026, 8:44:49 PM



File: Jorge Urias Phase 2 Feedback.docx

Here's the feedback

4/28/2026, 8:45:09 PM

I still can't see the contents of the feedback doc — only the file path is showing up, not the actual text. The .docx isn't extracting on my end.

Let me try reading it directly from the uploads folder.

```
Bash Tool: Check what's in uploads
```

Found it. Let me extract the text.

```
View: Check docx skill for how to read the feedback file
```

```
Bash Tool: Extract feedback text
```

Got it. Read the whole thing carefully. Here's my honest read.

86/100 is a strong score — up from 65 in Phase 1. The grader explicitly recognized that the Phase 2 submission addressed all three Phase 1 action items "precisely rather than generically," and called out the TC6 ablation and the Pydantic schema as the strongest pieces. The honest treatment of the TC2/TC6 result was specifically rewarded.

Now the part that matters: where the points were lost, and what Phase 3 needs to fix.

Where the 14 points went

Three categories had partial deductions, and there's a clear pattern:

Deduction	Category	Root cause
-2	Architecture	Custom orchestrator not named/justified vs. frameworks
-3	Tools/State	No per-agent read-access boundaries on RunState
-2	Prototype	No rendering branch yet (reserved for Phase 3)
-3	Evaluation	Only Agents 1-3 covered; preference test unrun; TC6 ran once vs. spec'd twice
-2	Risk/Governance	Disclaimer scene + per-scene citations planned but not implemented
-2	Documentation	Unchecked [] boxes in AI_USAGE.md Entry 02

Several of these are reserved for Phase 3 work to fill in (rendering branch, disclaimer scene). But three are immediately fixable or have a clear directive attached.

The grader's three explicit Phase 3 asks

The feedback ends with a "3 Things to Address" list, and these aren't suggestions — they're the criteria the same grader will use in Phase 3. Let me match them against your existing Phase 3 plan from `version_notes.md`:

1. Wire Agents 4a/4b and produce at least one rendered video. This was already item #3 on your `version_notes.md` list. Now it's not optional and not "scope down to one scene" — it's "minimum: TC1's verified plan through both agents + renderer for one scene, even if TTS/ffmpeg incomplete." The grader is being explicit about the minimum bar.

2. Apply v0.3 retrieval fixes and re-run TC2/TC5. This was item #1 on your list. The grader specifically called the v0.3 fix list (smaller chunks, higher top-k, query expansion) "specific and testable" — they're going to look for a before/after comparison. The grader also gave you a hedge: "If it only partially closes [the gap], document that honestly." Translation: partial success is fine, fabricated success is not.

3. Name and justify the orchestration approach. This is new — wasn't in your `version_notes.md` plan. The grader names LangGraph specifically as the obvious comparison for a verify-retry loop. Two options:

- Stay with the custom orchestrator and write a justification section comparing it to LangGraph (and probably one other framework like CrewAI or AutoGen)
- Migrate to LangGraph and document the migration

What this means for prioritization

Your Phase 3 list was retrieval → corpus stress test → rendering → end-to-end video. The feedback shifts this:

1. **Wire the rendering branch (4a + 4b + Renderer)** — non-negotiable, this is where the final artifact lives
2. **Re-run TC2/TC5 with v0.3 fixes** — the before/after evidence the rubric explicitly rewards
3. **Name and justify the orchestration approach** — could be a 1-2 page section in the Phase 3 report, or a migration to LangGraph
4. **Run the preference test with 5 reviewers** — was deferred in Phase 2, now needs to actually happen
5. **Run TC6 a second time** — methodology fix, cheap

Notice what dropped off: the "larger-corpus stress test" (your old item #2 to address F3). The grader didn't ask for it. They explicitly accepted the architecture-vs-baseline finding as written. Don't chase phantom requirements.

Three quick fixable issues

These can be done in under an hour total and recover small but easy points:

- **Check the unchecked boxes in AI_USAGE.md Entry 02** — if you've actually done those review steps, mark them `[x]`. If you haven't, do them now and then mark them.
- **Add per-agent read access to RunState** — the grader specifically called this out. Could be as simple as a `get_view_for_agent()` method that returns a read-only subset, or per-agent typed views via Pydantic.
- **Run TC6 a second time** — single API call, ~\$0.04, takes 20 seconds.

Worth doing before the bigger Phase 3 work because they require almost no thinking time.

A strategic question

Phase 3 is worth 25% of your final grade — almost twice Phase 2's weight. The rubric breakdown is different too:

- Final artifact quality: 20
- Evidence of agentic behavior: 15
- Evaluation quality: 15
- **Failure analysis and iteration: 15**
- Governance/responsible behavior: 10

- 5-min video: 10
- Documentation/portfolio quality: 15

That "failure analysis and iteration" 15-pointer is your strongest play given what you already have. You went into Phase 2 with three documented failures and a specific iteration plan. If v0.3 closes the retrieval gap (or partially closes it with honest documentation), that's the rubric's exact "what changed after testing" ask, executed cleanly.

So before we dive in:

What's your Phase 3 deadline, and what's your rough total time budget for the project? That determines whether we go for the full rendering pipeline (Animation Coder + Narration Writer + Renderer with TTS and ffmpeg) or scope to the grader's stated minimum (one rendered scene, no TTS/mux). The grader gave you permission to stop at "one rendered clip" — which is worth knowing.

4/28/2026, 8:46:00 PM

👤 Well I'm already past the due date technically. It was due Sunday and its Tuesday night. We have to work on Phase 3 and try to get it done by tonight or early morning tomorrow. In any case we can just get started. Let's work towards a working finished project. Our presentation is on Thursday, so we just need to focus on finishing the project first for now

4/28/2026, 8:55:02 PM

Got it. Phase 3 is overdue, presentation is Thursday, and the goal tonight is a finished, demoable project. Let me give you the honest read on what "finished" looks like given the time you have, then we move.

Triage: what's possible by tomorrow morning

You probably have ~6-10 hours of focused work, depending on when you sleep. Here's what fits and what doesn't:

Must-do for the rubric (non-negotiable, addresses grader's three asks):

1. Wire Agents 4a + 4b + run Renderer for at least one scene → produces a rendered clip
2. Re-run TC2 and TC5 with v0.3 retrieval fixes → before/after evidence
3. Name and justify orchestration approach → a section in the Phase 3 report

Should-do (recovers easy points): 4. Per-agent read access on RunState 5. Re-run TC6 a second time 6. Check unchecked boxes in AI_USAGE.md 7. Run preference test with reviewers (this is the time-risky one — needs other people)

Phase 3 deliverables (the actual submission package): 8. Final report PDF 9. Architecture diagram (already exists, may need update) 10. 4-8 screenshots with captions + screenshot_index.md 11. AI_USAGE.md updated for Phase 3 work 12. README updated for Phase 3 state 13. 5-minute video (this is for Thursday's presentation — but rubric also wants it submitted)

What I'd cut to save time:

- Full TTS + ffmpeg muxing into a final video file. The grader explicitly said "even if TTS and ffmpeg muxing are not complete for the full video" — one rendered Manim clip without audio is acceptable as the final artifact. You can show audio-less Manim renders in your video walkthrough and narrate over them.
- The preference test with 5 reviewers if you can't physically get reviewers tonight. Document it as "designed but not run due to time constraints" — honest is better than faked.

The order I'd attack this in

1. **Per-agent RunState access** (15 min) — quick win, locks in 1-2 points
2. **v0.3 retrieval fix + re-run TC2 and TC5** (45 min) — biggest leverage on the iteration narrative; this is the "what changed after testing" evidence
3. **Wire 4a (Animation Coder)** (60-90 min) — this is the longest pole; if it's going to break, it'll break here
4. **Render one scene with Manim** (30-45 min) — proves the artifact requirement
5. **Wire 4b (Narration Writer)** (30 min) — text output, much simpler
6. **Re-run TC6 + check AI_USAGE boxes** (20 min) — admin
7. **Final report + screenshots** (90-120 min) — the documentation pass
8. **Architecture diagram update + orchestration justification** (30 min)

That's roughly 6-7 hours of focused work if nothing major breaks. The 5-minute video is Thursday-morning work, not tonight's work.

Before we start, one quick decision:

Tool: Generating ask_user_input_v0...

My honest recommendations on each:

Orchestration: Keep custom and justify. Migrating to LangGraph at 11pm Tuesday with a Thursday presentation is high-risk for low marginal benefit. The grader explicitly said "either path is fine; the requirement is that the coordination layer is explicitly named and justified." A well-written justification is rubric-equivalent to a migration, and 2 hours saved is 2 hours you can spend on the rendering branch which actually shows up in the demo video.

Rendered video: Full Manim scenes for all of TC1's plan, no audio. One scene meets the minimum but looks thin in your presentation. TC1 has 5 scenes — rendering all 5 means you have ~30-60 seconds of actual video to show the grader, which makes the artifact feel real instead of token. Manim renders are deterministic; once one scene works, the others are mostly code generation. You can narrate over the silent video in your Thursday presentation.

Where to start: I'd actually suggest starting with **the v0.3 retrieval fix (#2) before per-agent RunState (#1)**. Reason: re-running TC2/TC5 with new config takes API time (~5 minutes wall-clock per run). If we kick those runs off early, they finish while you're working on the rendering branch. Per-agent RunState can slot in later — it's a code change, no API time involved.

Tap your picks and I'll start building. While you're tapping, one logistics question: do you have the project on this machine, or do you need me to work in this sandbox and ship you files to drop in? Last session you were running things locally at

`/Users/papijorge/Downloads/College/FourthYear/AgenticTech/Project/Phase2/Agentic Technologies Course Project/aurua`. I'll assume same setup unless you tell me otherwise.

4/28/2026, 8:55:38 PM

⊗ Q: On the orchestration justification — keep custom or migrate to LangGraph? A: Keep custom, write justification (faster; ~30 min)

Q: On the rendered video output — what's the minimum bar? A: Try full video + ffmpeg muxing + we can even try creating a script and then using eleven labs api for generating the voice and attach it to the video

Q: Want me to start building, or do you want to think about the plan first? A: Start now — pick up at #1 (per-agent RunState)

4/28/2026, 8:57:19 PM

Good. Let me push back on one of these before we start, because the time math matters.

You picked full video with TTS + ffmpeg + ElevenLabs. I have to be straight: that's the most ambitious option, and given you're starting at ~10pm Tuesday with a Thursday presentation, it has real risk. Here's my honest read:

What goes into "full video with audio":

- Manim scene rendering (1-3 hrs to debug first scene, then faster)
- Narration Writer agent producing timed scripts (30 min)
- ElevenLabs API integration + per-scene audio generation (30-45 min, plus you need an API key + credits)
- ffmpeg muxing video + audio with correct timing alignment (45-90 min — this is where it usually breaks)
- Inevitable debugging when timing doesn't match (30-90 min)

That's 4-7 hours just for the video. On top of: the v0.3 retrieval re-runs, per-agent RunState, the orchestration justification writeup, the final report, screenshots, README update, AI_USAGE update, and the architecture diagram update.

My counter-proposal: Build it in tiers, ship whichever tier you reach.

- **Tier 1** (must reach): Manim scenes rendered for TC1's verified plan, silent
- **Tier 2** (try if time): Narration script generated, ElevenLabs audio per scene, played alongside but not muxed
- **Tier 3** (stretch): ffmpeg muxes audio + video into one file

You build Tier 1 first as a complete, working artifact. If you have time, you add Tier 2 on top. If Tier 2 works, you try Tier 3. At any stop point, you have a finished deliverable.

This is a decision worth making before we start, not at 4am when you're staring at an ffmpeg sync bug.

```
Tool: Generating ask_user_input_v0...
```

Tap those and we move. Once I know the audio path I can lay out the build order properly.

While you decide: one thing I want to flag now so you're not surprised later. Your Phase 2 config in `.env.example` listed OpenAI as a TTS option but you never actually used it — the rendering branch was always Phase 3 work. If you go with OpenAI TTS, that's the cleaner path because it stays in the same SDK ecosystem you already have set up. If ElevenLabs, we add a new dependency and a new key. Both work, but OpenAI is one fewer integration point to debug at 2am.

Tap your picks.

4/28/2026, 8:57:40 PM

Q: Tiered build (Tier 1 → 2 → 3) or full send on the audio pipeline? A: Tiered approach — ship whichever tier I reach

Q: Do you have an ElevenLabs API key with credits already? A: I need to sign up / get one

4/28/2026, 8:58:35 PM

Good. Tiered approach with OpenAI TTS as the audio path is the smart call — you already have OpenAI set up from Phase 2, no new signup required, no credit purchase friction. ElevenLabs would have eaten 30-60 minutes on signup + key setup + first integration. We can still swap to ElevenLabs in Tier 2 if you finish Tier 1 with hours to spare.

Final build order, with time budget:

#	Task	Est. time	Tier
1	v0.3 retrieval fix + kick off TC2/TC5 re-runs	30 min	—
2	Per-agent RunState read access	20 min	—
3	Wire Agent 4a (Animation Coder) → produce Manim code	60 min	T1
4	Render TC1's scenes with Manim → silent .mp4 files	45 min	T1
5	Wire Agent 4b (Narration Writer) → timed scripts	30 min	T2
6	Generate per-scene audio with OpenAI TTS	30 min	T2
7	ffmpeg mux audio + video into final .mp4	60 min	T3
8	Re-run TC6, fix AI_USAGE checkboxes	15 min	—
9	Final report + screenshots + README + architecture	90 min	—
10	Orchestration justification section	30 min	—

Tier 1 is hours 1-4. Tier 2 hours 4-5. Tier 3 hour 5-6. Documentation hours 6-8. That's 8 hours total — tight but doable if we stay disciplined.

Now I need a few things from you before I can write code:

```
Tool: Generating ask_user_input_v0...
```

Tap those and we move. While you're answering, let me start drafting the v0.3 retrieval changes — that's the longest API-time task and the sooner we kick those re-runs off, the more they bake while we work on other things.

```
Bash Tool: Set up working directory in sandbox
```

I'll prep the v0.3 changes for `config.py` while you tap through those questions. Once I know your environment state I can adjust the file paths and tell you exactly what to drop where.

The three v0.3 retrieval fixes from `version_notes.md`:

- `CHUNK_SIZE_WORDS`: 80 → 40
- `CHUNK_OVERLAP_WORDS`: 20 → 15
- `RETRIEVAL_TOP_K`: 5 → 10
- New: query expansion in Agent 1 (synonyms + variant phrasings before FAISS search)

Tap the answers and I'll start producing actual code.

Q: Where is the aurua project right now — do you have it open? A: Yes, on this machine — I'll paste files/run commands

Q: Anthropic API key still working from Phase 2 runs? A: Yes, ANTHROPIC_API_KEY is in my .env

Q: Is Manim installed locally? (`pip show manim` to check) A: Not installed — will install now

4/28/2026, 8:59:21 PM

Good. Let me kick this off properly.

Step 1 — Install Manim (run this now in parallel)

Manim has system dependencies that take a few minutes to install on macOS. Start this **right now** in a separate terminal so it's done by the time we need it:

```
# In your project root, with your venv activated
brew install --cask mactex-no-gui # LaTeX for Manim text rendering — large download, ~2GB
brew install pkg-config cairo pango ffmpeg
pip install manim
```

If you're not on macOS or don't want the full LaTeX install, the lighter path is:

```
brew install pkg-config cairo pango ffmpeg
pip install manim
```

Manim will fall back to non-LaTeX text rendering, which is fine for our scenes — we're not rendering complex equations, mostly arrows, circles, and labels.

While that installs, let's do v0.3 retrieval changes. This is the highest-leverage piece because the re-runs need API time.

Step 2 — v0.3 retrieval config changes

Open `src/config.py` in your editor. Find these constants and change them:

```
# OLD (v0.2):
CHUNK_SIZE_WORDS = 80
CHUNK_OVERLAP_WORDS = 20
RETRIEVAL_TOP_K = 5

# NEW (v0.3):
CHUNK_SIZE_WORDS = 40
CHUNK_OVERLAP_WORDS = 15
RETRIEVAL_TOP_K = 10
```

That's it for `config.py` — three single-number changes.

Step 3 — Query expansion in Agent 1

This is the new piece. Currently Agent 1 (Intent & Grounding) takes the user question and embeds it directly. The grader's feedback specifically called out "query expansion" as one of the three v0.3 fixes. The idea: before retrieval, have Claude generate 2-3

paraphrased variants of the question, embed all variants, and union the top-k results.

Here's the patch. I need to see your current `src/agents/intent_grounding.py` first to make a precise edit. Can you paste it? Or, faster — run this and paste the output:

```
cd /path/to/aurua
cat src/agents/intent_grounding.py
```

While you grab that, here's what the change will look like in shape (you don't need to apply this yet — wait until I've adjusted to your actual code):

```
# New helper at top of intent_grounding.py
from src.claude_client import call_claude_text

def _expand_query(question: str) -> list[str]:
    """Generate 2-3 paraphrased query variants for retrieval coverage."""
    prompt = f"""Given this student question about a technical topic, generate exactly 3 paraphrased
    variants that capture different ways the same question might be asked. Focus on technical synonyms and
    variant phrasings.

    Original: {question}

    Return ONLY a JSON array of 3 strings, no other text. Example format:
    ["variant 1", "variant 2", "variant 3"]"""

    response = call_claude_text(prompt, model=cfg.MODEL_INTENT, max_tokens=300)
    variants = json.loads(_extract_json(response))
    return [question] + variants # Original + 3 variants = 4 queries
```

Then in the retrieval call inside Agent 1, instead of one query you do all four, dedupe by `span_id`, and take the top-k by max score.

Step 4 — While Manim installs, kick off the v0.3 re-runs

Once you've made the config changes (Step 2), do this:

```
# Clear the old FAISS index — it was built with chunk_size=80
rm -rf data/index/

# v0.3 retrieval re-run for TC2 (the primary anchor)
python -m src.run \
    --question "Why is the derivative of the cost with respect to a weight proportional to the
    activation feeding into it?" \
    --source data/transcripts/3b1b_ch4.txt

# v0.3 retrieval re-run for TC5 (multi-path)
python -m src.run \
    --question "Why does the chain rule give us a sum over paths when a neuron connects to multiple
    neurons?" \
    --source data/transcripts/3b1b_ch4.txt
```

Important: Don't add the query expansion yet. We do these runs first with **just the chunk size and top-k changes**, so we can isolate which fix moved the needle. If TC2 passes with just chunking changes, query expansion becomes a bonus rather than required. If it doesn't, we add query expansion and re-run.

This gives you cleaner before/after evidence: "v0.3a (chunk + top-k) → result. v0.3b (+ query expansion) → result." Two data points, one for each fix family.

What to do right now, in order

1. Start Manim install in a separate terminal (it'll run for 5+ minutes)
2. Edit `src/config.py` — three numbers
3. `rm -rf data/index/`
4. Run the v0.3 TC2 command above
5. Paste me the console output + the new `state.json` from `outputs/runs/<new_run_id>/`
6. Also paste me your current `src/agents/intent_grounding.py` so I can write the query expansion patch precisely

While the runs go, I'll start on the per-agent RunState read access and the Animation Coder agent skeleton in the sandbox here. By the time you come back with TC2 results, I'll have those files ready for you to drop in.

Go.

4/28/2026, 8:59:50 PM



File:

Here's my project directory tree

4/28/2026, 9:01:06 PM

Good. I have a clear picture of your project state. A few things I notice:

1. **Architecture diagram is a PDF, not editable** — we'll need to update it for Phase 3 (add the rendering branch). I'll regenerate the source HTML.
2. `docs/~$rge Urias Phase 2 Feedback.docx` — that's a Word lock file (Office leaves these when a doc is open). Close that doc in Word at some point or `rm` the lock file. Not urgent.
3. **No `.venv` shown** — assuming you have one at the project root not displayed by the tree depth.
4. **Existing prompts for Animation Coder + Narration Writer** — `src/prompts/animation_coder.py` and `src/prompts/narration.py` already exist as scaffolds. We'll fill those in.
5. **Existing narration_writer agent stub** — `src/agents/narration_writer.py` exists but probably empty/stub from the Phase 2 scaffold.

Now I need to see your current files so I can write precise patches instead of guessing. Paste me the contents of these four files (you can do it in one message or split across messages):

```
# Run all four and paste the output
cat src/config.py
echo "=====
cat src/agents/intent_grounding.py
echo "=====
cat src/state.py
echo "=====
cat src/run.py
```

While you grab those, let me also start Manim install instructions cleaner so you don't waste time on the wrong path.

Manim install — pick the lighter path

The full LaTeX install (`mactex-no-gui`) is 2GB and takes 10+ minutes. You don't need it for the scenes we'll write. Run this instead:

```
brew install pkg-config cairo pango ffmpeg
pip install manim
```

That's enough for `Text`, `Circle`, `Arrow`, `Line`, `MathTex` (which needs LaTeX), and basic geometry. We'll avoid `MathTex` and use `Text` for everything — slightly less pretty math rendering but works without LaTeX, and the grader cares about the architecture and grounding, not whether `dc/dw` looks like proper math typography.

Actually — one quick verification. Run this to check what's already there:

```
which ffmpeg
which python
python -c "import sys; print(sys.version)"
```

If `ffmpeg` exists already, skip the brew step. Paste the output.

Order of operations from here

1. You paste me those 4 files + the brew/which output
2. I write precise patches for `config.py` (v0.3 numbers) and `intent_grounding.py` (query expansion)
3. You apply the patches, `rm -rf data/index/`, run TC2 + TC5 v0.3
4. While those run, I write the Animation Coder agent + a working Manim scene template
5. We iterate from there

I'm not going to write speculative code without seeing your actual files first — Phase 2 already taught us that the prompt names, function signatures, and import patterns matter and I'd rather get the patches right the first time than have you debug imports at midnight.

Paste those four files.

4/28/2026, 9:01:29 PM



File:

4/28/2026, 10:31:53 PM

Perfect. Now I have what I need. Two things to flag before writing patches:

1. **Constant name correction.** Your `config.py` uses `TOP_K_RETRIEVAL`, not `RETRIEVAL_TOP_K` like I said earlier. I had it wrong. The patches below use the correct name.
2. **Manim install.** Hold off on `mactex-no-gui`. Just run `brew install pkg-config cairo pango ffmpeg` then `pip install manim` — that's enough for our scenes.

Let's go. I'll give you everything in order so you can apply patches and kick off the v0.3 runs while I build the rendering branch in the sandbox.

Patch 1: `src/config.py`

Replace the `# --- Retrieval ---` block with this:

```
# --- Retrieval ---
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE_WORDS = 40          # v0.3: was 80, smaller chunks improve coverage
CHUNK_OVERLAP_WORDS = 15      # v0.3: was 20, proportional to new chunk size
TOP_K_RETRIEVAL = 10          # v0.3: was 5, more candidates compensate for chunking gaps

# Query expansion (v0.3): generate paraphrased variants of the user question
# before retrieval, to address F1/F2 retrieval-coverage failures from Phase 2.
ENABLE_QUERY_EXPANSION = True
QUERY_EXPANSION_VARIANTS = 3
```

Patch 2: `src/agents/intent_grounding.py`

This adds query expansion as a pre-retrieval step. The variants get embedded, all queries hit FAISS, results are deduplicated by `span_id`, and the top-k by max score across queries goes to Claude. Replace your entire `intent_grounding.py` with this:

```
"""Agent 1: Intent & Grounding.

v0.3: Adds query expansion before retrieval to address Phase 2 F1/F2
retrieval-coverage failures. The user's question is paraphrased into
N variants by Claude; each variant is embedded and used as a FAISS
query; results are deduplicated by span_id and ranked by max score.
"""

from __future__ import annotations

import json
from pathlib import Path

from ..claude_client import call_claude_structured
from ..prompts.intent import INTENT_SYSTEM_PROMPT
```

```

from ..state import IntentOutput, RunState, RunStatus
from ..tools.retrieval import Retriever
from .. import config

# -----
# Query expansion (v0.3)
# -----

from pydantic import BaseModel

class _QueryVariants(BaseModel):
    variants: list[str]

_QUERY_EXPANSION_SYSTEM = """You are a query expansion assistant for a retrieval system.

Given a student's question about a technical topic, generate paraphrased
variants that capture different ways the same question might be asked. Use
technical synonyms, alternative phrasings, and rephrase from different angles
(definition, mechanism, justification, example).

Return ONLY a JSON object with this exact schema:
{"variants": ["variant 1", "variant 2", "variant 3"]}

Rules:
- Generate exactly the number of variants requested.
- Each variant must be a complete question, not a phrase.
- Each variant must be semantically equivalent to the original – no new topics.
- Vary vocabulary aggressively. If the original uses "derivative", a variant might use "rate of
change" or "sensitivity".
"""

def _expand_query(question: str, state: RunState) -> list[str]:
    """Return [original] + N paraphrased variants. On failure, returns [original]."""
    if not config.ENABLE_QUERY_EXPANSION:
        return [question]

    user_prompt = (
        f"Original question: {question}\n\n"
        f"Generate exactly {config.QUERY_EXPANSION_VARIANTS} paraphrased variants."
    )

    trace_dir = Path(config.TRACES_DIR) / state.run_id

    try:
        result = call_claude_structured(
            system_prompt=_QUERY_EXPANSION_SYSTEM,
            user_prompt=user_prompt,
            response_model=_QueryVariants,
            cost_tracker=state.cost,
            trace_label="00_query_expansion",

```

```

        trace_dir=trace_dir,
    )
    return [question] + result.variants
except Exception as exc: # noqa: BLE001
    # Don't fail the pipeline over query expansion – fall back to single query.
    print(f"[query expansion failed: {exc}] – falling back to original question only")
    return [question]

# -----
# Multi-query retrieval
# -----
def _multi_query_retrieve(retriever: Retriever, queries: list[str], top_k: int) -> list:
    """Run retrieval for each query, dedupe by span_id, rank by max score."""
    seen: dict[str, object] = {} # span_id -> hit (with the highest score so far)
    for q in queries:
        for hit in retriever.search(q, top_k=top_k):
            sid = hit.chunk.chunk_id
            if sid not in seen or hit.score > seen[sid].score:
                seen[sid] = hit
    # Sort by score descending, take top_k overall
    ranked = sorted(seen.values(), key=lambda h: h.score, reverse=True)
    return ranked[:top_k]

# -----
# Main agent entry point
# -----
def run_intent_agent(state: RunState, retriever: Retriever) -> RunState:
    """Retrieve spans (with query expansion), then identify confusion + key claims."""
    # 1. Query expansion (v0.3)
    queries = _expand_query(state.user_question, state)

    # 2. Multi-query retrieval with dedup
    hits = _multi_query_retrieve(retriever, queries, top_k=config.TOP_K_RETRIEVAL)

    retrieved_spans_for_prompt = [
        {
            "span_id": hit.chunk.chunk_id,
            "text": hit.chunk.text,
            "location": hit.chunk.location,
            "retrieval_score": round(hit.score, 4),
        }
        for hit in hits
    ]

    # 3. Ask Claude to structure the intent
    user_prompt = (
        f"Student's question:\n{state.user_question}\n\n"
        f"Retrieved source spans (top {len(hits)} after query expansion + dedup):\n"
        f"{json.dumps(retrieved_spans_for_prompt, indent=2)}\n\n"
        f"Produce the JSON described in your instructions."
    )

```

```

trace_dir = Path(config.TRACES_DIR) / state.run_id

intent = call_claude_structured(
    system_prompt=INTENT_SYSTEM_PROMPT,
    user_prompt=user_prompt,
    response_model=IntentOutput,
    cost_tracker=state.cost,
    trace_label="01_intent",
    trace_dir=trace_dir,
)

state.intent_output = intent

if intent.confusion_type == "vague":
    state.status = RunStatus.CLARIFICATION_REQUIRED
else:
    state.status = RunStatus.INTENT_COMPLETE

return state

```

Patch 3: requirements.txt

Uncomment the rendering deps:

```

# Core LLM SDKs
anthropic>=0.39.0
openai>=1.50.0

# Retrieval
faiss-cpu>=1.8.0
sentence-transformers>=3.0.0
numpy>=1.26.0

# Utilities
python-dotenv>=1.0.0
pydantic>=2.9.0

# Rendering (Phase 3)
manim>=0.18.0

# Dev
pytest>=8.3.0

yt-dlp==2026.03.17

```

(Note: dropped `ffmpeg-python` — we'll call `ffmpeg` via subprocess instead, simpler and one less dep.)

Run sequence

Apply patches 1, 2, 3. Then in your terminal:

```
# Verify syntax before running
python -c "from src.agents.intent_grounding import run_intent_agent; print('ok')"
```



```
# Clear stale FAISS index (was built with chunk_size=80)
rm -rf data/index/
```



```
# v0.3 retrieval re-run for TC2 (the primary anchor that exhausted in Phase 2)
python -m src.run \
  --question "Why is the derivative of the cost with respect to a weight proportional to the
activation feeding into it?" \
  --source data/transcripts/3b1b_ch4.txt
```

Expected: ~3-7 API calls (query expansion + intent + planner + verifier loop), \$0.05-0.15, 30-60 seconds. The interesting bit is whether the verifier passes this time — if it does, you have your before/after evidence locked in.

Paste me the console output. Don't run TC5 yet — let's confirm v0.3 actually moves the needle on TC2 before re-running TC5. If TC2 still exhausts, we adjust before burning another run.

While you run that and the Manim brew install completes, I'll start building the Animation Coder agent + a working Manim scene template in the sandbox. By the time you come back with TC2 v0.3 output, that'll be ready.

Go.

4/28/2026, 10:32:37 PM

👤 remind me why we are changing `config.py` and `intent_grounding.py`?

4/28/2026, 10:38:17 PM

Good question to stop and ask — make sure you understand why before applying changes.

Both changes address the **F1 / F2 retrieval-coverage failures** documented in your Phase 2 `failure_log.md`. Here's the story.

What happened in Phase 2

On TC2, the question was "*why is the derivative of the cost with respect to a weight proportional to the activation feeding into it?*" The answer is literally on line 63 of `3b1b_ch4.txt`: "*The derivative of $z(L)$ with respect to $w(L)$... comes out to be $a(L-1)$.*"

But FAISS never retrieved that span. It returned 5 chunks, none of which contained the line-63 answer. The Planner was forced to work with weaker material, made claims it couldn't ground, the Verifier rejected them, and the loop exhausted at 7 API calls without producing a verified plan.

TC5 had the same pattern — the answer was at lines 115-119, retrieval missed it, verifier exhausted.

The TC6 ablation made this concrete: the single-prompt baseline (which had the *whole transcript* in context, no retrieval) produced a fully grounded plan in 1 API call citing line 63 correctly. The multi-agent system was *worse than the baseline* on this case, and the cause traced specifically to retrieval missing the right span.

Why each change

Change 1: smaller chunks (`CHUNK_SIZE_WORDS: 80 → 40`)

With 80-word chunks, line 63's key fact (" $dz/dw = a(L-1)$ ") got bundled with 79 other words. The chunk's overall semantic vector is a soup of everything in those 80 words, which dilutes the match against a focused query. Smaller chunks mean each chunk is "about" fewer things, so a chunk containing the key claim has a vector that points more directly at that claim.

Change 2: more candidates (`TOP_K_RETRIEVAL: 5 → 10`)

Even with better chunking, the right chunk might rank #6 or #7 instead of #1. Top-5 was too aggressive a filter — by retrieving 10 instead, we give the Planner more material to work with. The cost is some noise (less-relevant chunks), but the Planner can ignore those; what it can't do is ground claims in a chunk it never saw.

Change 3: query expansion in `intent_grounding.py`

This is the bigger architectural change. Right now retrieval embeds *one* phrasing — the user's exact question — and matches it against chunks. But the answer in the transcript might be phrased completely differently. The user asks about "*derivative... proportional to the activation*"; the transcript says "*comes out to be $a(L-1)$* ". Same concept, almost no shared vocabulary. Embedding similarity catches some of this but not all.

Query expansion has Claude generate 3 paraphrased variants of the question first ("how does the previous neuron's activation determine the weight's gradient?", "what makes dC/dw scale with the upstream value?", etc.). All 4 queries (original + 3 variants) hit FAISS. Results are deduplicated by `span_id` and ranked by max score across queries. The chunk that line 63 lives in might score 0.45 against the original question but 0.78 against a paraphrase that uses different words — and that's the score that gets it into the top 10.

Why these are the *right* fixes

These three changes were specifically named in `eval/VERSION_NOTES.md` as the v0.3 plan, written when you were analyzing the Phase 2 failures. The Phase 2 grader read that plan in your version notes and called it "*specific and testable*" — it was identified in the feedback

as one of the three things they want to see executed in Phase 3.

So: not a guess, not new architecture invented tonight. These are the planned fixes from your own failure analysis, and the grader is expecting before/after evidence on them.

Why this matters for the rubric

Phase 3's "Failure analysis and iteration" category is worth 15 points. The rubric explicitly asks for "*what changed after testing.*" Your Phase 2 left you with two documented retrieval failures and a specific fix list. If TC2 and TC5 pass after v0.3 changes — or even partially pass — that's the cleanest possible "before / after" evidence for the rubric. It's the highest-leverage thing you can do tonight, which is why we're starting here.

Worth noting honestly

These changes might *not* fully fix TC2/TC5. The Phase 2 ablation revealed that retrieval at this corpus size (1,600 words) is fundamentally over-engineered — even with perfect retrieval, you're chunking material that fits in a single context window. The grader acknowledged this and gave you a hedge: "*if v0.3 only partially closes the gap, document that honestly.*"

So we run v0.3, see what happens, and write up whatever the result is. Either outcome is rubric-compliant as long as we don't fake the numbers.

Make sense? Want to go ahead with the patches, or any concerns first?

4/28/2026, 10:38:50 PM



File: state.json

File: 00_query_expansion.txt

File: 01_intent.txt

File: 02_planner_initial.txt

File: 02_planner_retry_0.txt

File: 02_planner_retry_1.txt

File: 03_verifier_attempt_0.txt

File: 03_verifier_attempt_1.txt

File: 03_verifier_attempt_2.txt

Output:

Verify syntax before running —

```
python -c "from src.agents.intent_grounding import run_intent_agent; print('ok')"
```

Clear stale FAISS index (was built with chunk_size=80)

```
rm -rf data/index/
```

v0.3 retrieval re-run for TC2 (the primary anchor that exhausted in Phase 2)

```
python -m src.run
```

```
--question "Why is the derivative of the cost with respect to a weight proportional to the activation feeding into it?"
```

```
--source data/transcripts/3b1b_ch4.txt ok Loading weights: 100%|
```

```
103/103 [00:00<00:00, 16147.01it/s] BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2 Key | Status | | -----
-----+-----+--+ embeddings.position_ids | UNEXPECTED | | Notes:
```

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch. Run ID: c3061f8c2527 Status: verification_exhausted Retries used: 2 Cost: 13203 in, 3842 out across 8 calls (50.67s) Final plan (3 scenes, 6 total): [Scene 1] The weighted sum $z = w \cdot a_{\text{prev}} + b$ is a linear function of w , so $\partial z / \partial w = a_{\text{prev}}$ (the activation feeding into the weight). visual: Show two labeled circles (neurons) connected by an arrow labeled ' w '. Display the equation $z = w \cdot a_{\text{prev}} + b$ as a text label near the connection. Highlight w in one color and a_{prev} in another. Then animate a small nudge Δw to w and show the resulting change in z equals $a_{\text{prev}} \Delta w$, making clear the slope is a_{prev} . source: c6e5a48a3e (20s) [Scene 2] By the chain rule, $\partial C / \partial w = (\partial C / \partial a) \cdot (\partial a / \partial z) \cdot (\partial z / \partial w)$, and since $\partial z / \partial w = a_{\text{prev}}$, the entire gradient is proportional to that incoming activation. visual: Display a horizontal chain of three labeled boxes: ' $w \rightarrow z \rightarrow a \rightarrow C$ ', connected by arrows. Below each arrow, show the corresponding partial derivative label: $\partial z / \partial w$, $\partial a / \partial z$, $\partial C / \partial a$. Then animate the chain rule product assembling left-to-right, and finally substitute $\partial z / \partial w = a_{\text{prev}}$ so the full expression reads $\partial C / \partial w = (\partial C / \partial a) \cdot (\partial a / \partial z) \cdot a_{\text{prev}}$, with a_{prev} highlighted. source: 5cc48e0882 (25s) [Scene 3] A larger incoming activation means a nudge to the weight has a larger effect on the weighted sum, and therefore larger effect on the cost — this is the 'neurons that fire together wire together' principle. visual: Show two side-by-side comparisons: on the left, a dimly colored source neuron (small a_{prev}) connected by an arrow to a target neuron, with a small bar indicating the resulting gradient magnitude; on the right, a brightly colored source neuron (large a_{prev}) with a proportionally

taller bar. Animate both bars growing simultaneously to contrast them. Add a text label 'neurons that fire together wire together' fading in beneath the comparison. source: c6e5a48a3e (20s) Full state saved to: /Users/papijorge/Downloads/College/FourthYear/AgenticTech/Project/Phase2/Agentic Technologies Course Project/aurua/outputs/runs/c3061f8c2527/state.json Traces saved to: /Users/papijorge/Downloads/College/FourthYear/AgenticTech/Project/Phase2/Agentic Technologies Course Project/aurua/eval/traces/c3061f8c2527/

4/28/2026, 10:45:29 PM

Important data. Let me read this carefully — because the headline ("verification_exhausted, same as before") hides what actually changed, and one of those changes matters a lot.

What actually moved

Retrieval IS dramatically better. Compare TC2 v0.2 vs v0.3:

Span	v0.2 top-5?	v0.3 top-10?	What it contains
L66-69 (c6e5a48a3e)	✗ Not retrieved	✓ Rank ~8, score 0.624	The "neurons fire together" answer
L37-40 (2ebe998091)	✗ Not retrieved	✓ Rank 4, score 0.6726	Sensitivity goal
L110-113 (5cc48e0882)	✗ Not retrieved	✓ Rank 1, score 0.7154	Chain-rule reference
L75-79 (01351e13bb)	✗ Not retrieved	✓ Rank 5, score 0.6685	"More than 50% of the work"

In v0.2, top-5 surfaced spans like L34-41, L83-89 (wrong derivative — dz/da not dz/dw), L110-117. In v0.3, **all four spans the Intent agent identified as relevant are now retrieved and used.** That's the retrieval fix working exactly as designed.

The query expansion also worked beautifully. Look at [00_query_expansion.txt](#) — Claude generated three solid paraphrases that vary vocabulary aggressively ("rate of change of the error", "sensitivity of the cost function to a particular weight"). Those drove the embedding diversity.

But there's still a critical gap: the span at line 63 — *"The derivative of $z(L)$ with respect to $w(L)$... comes out to be $a(L-1)$ "* — is **still not retrieved**, even with v0.3. That specific line is the smoking-gun answer. The transcript chunks are now smaller (40 words), but that line apparently sits in a chunk whose semantic vector still doesn't match well against the question phrasings.

So why did verification still exhaust?

Look at the verifier's complaint across all three attempts. It's the same one each time, on Scene 2:

"The cited span only references a 'chain-ruled derivative expression' and says it 'looks essentially the same' — it does not explicitly state the chain rule product $\partial C/\partial w = (\partial C/\partial a) \cdot (\partial a/\partial z) \cdot (\partial z/\partial w)$ nor the substitution $\partial z/\partial w = a_{prev}$. The claim requires explicit derivation steps that are absent from this span."

The verifier is right. None of the four retrieved spans contain the explicit chain rule decomposition. Line 63 has the substitution piece ($\partial z/\partial w = a_{prev}$) but it's not in the retrieved set. The actual chain rule product $(\partial C/\partial a) \cdot (\partial a/\partial z) \cdot (\partial z/\partial w)$ is at lines ~52-53 in the transcript, and that span also wasn't retrieved.

The Planner is stuck: it knows the right answer (because Claude has internal knowledge of backprop), it makes a true claim, but it can't ground that claim in any retrieved span. The Verifier correctly refuses to approve it. The Planner cycles between two non-supporting spans ([5cc48e0882](#) and [2ebe998091](#)), neither of which contain the explicit derivation.

This is a **different failure mode** than Phase 2:

- Phase 2 TC2: retrieval missed the answer entirely; verifier rejected because spans were on wrong topic (dz/da vs dz/dw).
- Phase 3 TC2 v0.3: retrieval surfaces *related* spans, planner makes a true claim, verifier correctly notes the explicit derivation isn't in any retrieved span.

What this means for the rubric

This is actually a richer story than "v0.3 fixed it." Read what the grader explicitly told you:

"If v0.3 only partially closes [the gap], document that honestly."

You now have:

1. **Retrieval coverage measurably improved** — 4/4 of the Intent agent's chosen spans were unretrievable in v0.2 and now all rank in the top-10. Quantifiable progress.
2. **Verifier remains correctly strict** — even given more material, it won't approve a claim that isn't explicitly supported by the cited span. This is the verifier doing its job, not failing.
3. **A new finding:** retrieval-coverage improvement is necessary but not sufficient. The transcript itself is informal — Grant Sanderson never writes the chain rule decomposition out as a formal product. He gestures at it ("looks essentially the same"). The Planner trying to formalize an informal source will always fail strict grounding.

That third point is a *genuine architectural insight* you discovered through evaluation. It's exactly what the Phase 3 rubric's "Failure analysis and iteration" 15-pointer is asking for.

Three honest options going forward

I'd push hard on option 2, but you should know all three:

Tool: Generating ask_user_input_v0...

My honest recommendation: **option 1** — use TC1 as the demo anchor.

Here's why:

- **TC1 already verifies cleanly** in your Phase 2 archive ([eval/runs_saved/phase2_evaluation/TC1_verified_retry1/](#)). 5 scenes, verifier caught real planner overreach on attempt 0, planner revised, verifier passed on attempt 1. *That* is the canonical demo of the architecture working as designed.
- **TC2's story becomes the iteration narrative**, not the demo. Phase 2: retrieval missed the right spans. Phase 3 v0.3: retrieval improved 4× but exposed a deeper finding — the source itself doesn't contain formal derivations. This is the exact "what changed after testing" arc the rubric wants.
- **Option 2 (relaxing the verifier) is risky.** The verifier's strictness IS the value proposition. Loosening it to make TC2 pass is exactly the kind of fudge that would tank the governance/trust 10-pointer if a grader noticed. We can't show off the verifier catching hallucinations and *also* tune it down to silence valid rejections.
- **Option 3 is reasonable but burns time.** We'd need to rerun the test case selection process. We're already past the deadline — staying with what works gets you to a demoable artifact faster.

Concrete plan if you go with option 1:

1. Re-run TC1 with v0.3 to get a fresh state.json showing the retrieval improvements still help when the case verifies. (~30 sec)
2. Render TC1's verified plan with the Animation Coder + Manim → silent video
3. Generate narration with OpenAI TTS, mux with ffmpeg → final video

4. Document TC2 v0.3 as a "partial fix with deeper finding" — this becomes one of your two failure cases in `failure_log.md`

Does this match where you're at? Tap option 1 (or another) and I'll start building the Animation Coder while you re-run TC1.

Also: while I have you waiting, you should kick off the v0.3 TC1 run **right now** so it's done by the time we need it. Run this in another terminal:

```
python -m src.run \  
  --question "What does the gradient of the cost function tell us about which weights to change?" \  
  --source data/transcripts/3b1b_ch3.txt
```

And paste the run_id when it finishes. That's our demo plan.

4/28/2026, 10:46:20 PM



File: state.json

File: 00_query_expansion.txt

File: 01_intent.txt

File: 02_planner_initial.txt

File: 02_planner_retry_0.txt

File: 02_planner_retry_1.txt

File: 03_verifier_attempt_0.txt

File: 03_verifier_attempt_1.txt

File: 03_verifier_attempt_2.txt

Output:

```
python -m src.run \ --question "What does the gradient of the cost function tell us about which weights to change?"  
--source data/transcripts/3b1b_ch3.txt Loading weights: 100%|
```

```
103/103 [00:00<00:00, 10126.89it/s] BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2 Key | Status | | -----  
-----+-----+----- embeddings.position_ids | UNEXPECTED | | Notes:
```

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch. Run ID: 729f60aeae1 Status: verification_exhausted Retries used: 2 Cost: 14167 in, 4255 out across 8 calls (62.41s) Final plan (4 scenes, 8 total): [Scene 1] The gradient of the cost function is a vector made up of the partial derivatives of the cost with respect to every weight and bias in the network. visual: Show a small neural network diagram with labeled weights (w_1, w_2, w_3) and biases (b_1, b_2, b_3). Fade in a column vector on the right side of the screen labeled 'gradient' with entries $\partial C / \partial w_1, \partial C / \partial w_2, \partial C / \partial w_3, \partial C / \partial b_1, \partial C / \partial b_2, \partial C / \partial b_3$. Draw arrows from each weight/bias label in the network to its corresponding row in the vector to show the one-to-one correspondence. source: 0b4ea9579c (20s) [Scene 2] Each partial derivative measures how sensitive the cost function is to a small change in one specific weight — i.e., how much nudging that weight moves the cost up or down. visual: Zoom in on a single weight $w(L)$ and its number line. Show a small rightward nudge (Δw) on the number line, then animate an arrow pointing to a separate cost axis showing the resulting small change in cost (ΔC). Label the ratio $\Delta C / \Delta w$ as the partial derivative. Use color highlight on $w(L)$ in the network diagram to keep context clear. source: 2ebe998091 (20s) [Scene 3] The gradient vector is built from partial derivatives of the cost with respect to all weights and biases, and knowing these tells us which adjustments will cause the most efficient decrease to the cost function. visual: Display the gradient vector from scene 1. Animate each entry lighting up one at a time with a small bar beside it showing a sample magnitude value. Annotate that a large entry means that weight has a large influence on the cost. Add text label: 'Larger partial derivative → bigger effect on cost → higher priority to adjust.' Keep the neural network diagram visible in the corner for context. source: 4510ebfb14 (20s) [Scene 4] Knowing how sensitive the cost function is to each weight and bias tells us which adjustments to those terms will cause the most efficient decrease to the cost function. visual: Show a simple 2D cost landscape drawn as concentric ellipses (contour lines) with a dot representing current weights. Highlight the gradient vector entries from the previous scene and show a text annotation: 'Adjust weights most where sensitivity is highest.' Animate the dot taking two steps toward the center of the ellipses, with each step labeled as moving in the direction that reduces cost. Keep the visual minimal: dot, ellipses, step arrows, and text labels only. source: 448484cf9c (25s) Full state saved to: /Users/papijorge/Downloads/College/FourthYear/AgenticTech/Project/Phase2/Agentic Technologies Course Project/aurua/outputs/runs/729f60aeae1/state.json Traces saved to: /Users/papijorge/Downloads/College/FourthYear/AgenticTech/Project/Phase2/Agentic Technologies Course Project/aurua/eval/traces/729f60aeae1/

Wait — read your command line carefully. You ran TC1 against `3b1b_ch4.txt`, not `3b1b_ch3.txt`.

```
--source data/transcripts/3b1b_ch3.txt
```

That's what you typed, but look at the retrieved spans in the trace — every single one is from `3b1b_ch4.txt`:

```
"location": "3b1b_ch4.txt:L75-L79"  
"location": "3b1b_ch4.txt:L110-L113"  
"location": "3b1b_ch4.txt:L37-L40"
```

This is the FAISS index issue I flagged earlier in the conversation but didn't make explicit enough: **the index doesn't track which file each chunk came from for the source you specified at runtime**. When you ran TC2 first, the index was built from `3b1b_ch4.txt`. Then you `rm -rf data/index/` was supposed to clear it — but did it? Let me check, because if the index persisted, TC1's `--source 3b1b_ch3.txt` got ignored and Ch4 chunks were searched.

Actually no, looking at your run sequence — you only `rm -rf`'d once before TC2. Between TC2 and TC1 you didn't clear it. So this TC1 run was searching the Ch4 index. That's the bug.

This matters for two reasons:

1. **The original Phase 2 TC1 was on Ch3.** The conceptual question about gradient meaning is anchored in Chapter 3's intuitive material. Running it against Ch4 (the calculus chapter) gives the verifier a harder time because Ch4 is more about the mechanics than the conceptual intuition.
2. **The verifier was correctly strict here too** — same pattern as TC2. Look at the rejections: the Planner kept trying to ground "gradient descent" / "negative gradient" / "follow the negative gradient to minimize cost" claims in Ch4 spans that talk about *sensitivity* but never explicitly mention gradient descent. Because Ch4 doesn't really teach gradient descent — Ch3 does.

So this is the same root cause again: the planner is making true claims that aren't explicitly stated in the cited Ch4 span, because the conceptual material lives in Ch3.

Two paths forward

Both involve a clean re-run, but they differ in what they're testing:

```
Tool: Generating ask_user_input_v0...
```

My strong recommendation: **option 1** — re-run TC1 against Ch3, which is what TC1 was designed for. Phase 2 archive shows this run already passed with 5 scenes (verifier caught real overreach on attempt 0, planner revised, verifier passed on attempt 1). Should reproduce cleanly.

Run this:

```
# Critical: clear the stale Ch4 index first  
rm -rf data/index/  
  
# TC1 against the CORRECT source (Ch3, not Ch4)  
python -m src.run \  
  --question "What does the gradient of the cost function tell us about which weights to change?" \  
  --source data/transcripts/3b1b_ch3.txt
```

Watch for two things in the output:

1. The retrieved span locations should be `3b1b_ch3.txt:L*` — not Ch4.
2. The status should be `verified` (ideally on attempt 1 like Phase 2).

While that runs, I'm going to start writing the Animation Coder agent in the sandbox so it's ready when TC1 verifies.

```
Bash Tool: Check available skills
```

None of the available skills cover Manim code generation, so I'll write the agent directly. Now in the sandbox, let me draft the Animation Coder.

```
Create File: Draft the Animation Coder agent
```

That references `call_claude_text` which doesn't exist yet in your `claude_client.py` — only `call_claude_structured` does. So I need to know what's in your client wrapper before I can finalize this. While you re-run TC1 with the right source, paste me:

```
cat src/claude_client.py
cat src/prompts/animation_coder.py
```

Two reasons I need both:

1. `claude_client.py` — to see if there's already a text-mode call function or if I need to add one. The Animation Coder needs raw text output (Manim code), not a Pydantic-validated structured response.
2. `prompts/animation_coder.py` — your Phase 2 scaffold had a stub here. I want to see what's there so I either build on it or replace it cleanly.

So while TC1 v0.3 runs against Ch3, paste those two files, and I'll have the full Animation Coder + Manim integration ready when TC1 verifies.

Also, since the sandbox can't actually run your project, I should be straight: I'll be drafting code for you to drop in. Every patch I write needs to be tested on your machine. We'll iterate fast — I write, you paste, you test, you paste back what broke.

Go.

4/28/2026, 10:50:45 PM

Q: What's the move? A: Re-run TC1 against the correct source (Ch3) — likely passes, becomes the demo

4/28/2026, 10:51:20 PM

Good. Run this now while I keep building:

```
rm -rf data/index/

python -m src.run \
  --question "What does the gradient of the cost function tell us about which weights to change?" \
  --source data/transcripts/3b1b_ch3.txt
```

Confirm in the output that retrieved spans show `3b1b_ch3.txt:L*` (not Ch4). Paste me the console output.

While that runs, I still need those two files to build the Animation Coder cleanly. Run these and paste:

```
cat src/claude_client.py
echo "===== "
cat src/prompts/animation_coder.py
```

Reasons I need both:

- `claude_client.py` — your Phase 2 had `call_claude_structured` which validates against a Pydantic model. The Animation Coder needs raw text output (Python code, not JSON). I need to see if a text-mode helper already exists or if I need to add one alongside it.
- `prompts/animation_coder.py` — the Phase 2 scaffold left a prompt stub here. If it's already drafted I'll build on it; if it's just a placeholder I'll write a fresh prompt that handles the no-LaTeX constraint, the "no MathTex" rule, and the runnable-class-name pattern (`Scene01`, `Scene02`, etc.).

Once I have those two files, the next batch I send you will be:

1. `src/claude_client.py` patch — adds `call_claude_text()` for raw text mode
2. `src/prompts/animation_coder.py` — full prompt with Manim constraints
3. `src/agents/animation_coder.py` — the agent itself (skeleton already drafted in my sandbox)
4. A patch to `src/run.py` — adds the rendering branch after VERIFIED
5. A working Manim test scene — so we can verify your install works *before* generating real scenes

Paste those files and the TC1 output. Three things in flight at once is the right pace right now.

4/28/2026, 10:51:37 PM



File: state.json

File:

File: 00_query_expansion.txt

File: 01_intent.txt

File: 02_planner_initial.txt

File: 03_verifier_attempt_0.txt

File:

Here's what I got

4/28/2026, 10:53:48 PM

Excellent. **TC1 verified first-try, 4 calls, \$0.07, 30 seconds, all 4 scenes grounded with confidence 0.85-0.95.**

This is the canonical demo — better than the Phase 2 v0.2 run, which needed a retry. The query expansion + smaller chunks pulled in stronger spans (top 3 retrieval scores 0.72/0.72/0.72 vs Phase 2's 0.68 ceiling), so the planner had better material to ground against.

Save this run before anything else:

```
mkdir -p eval/runs_saved/phase3_evaluation
cp -r outputs/runs/fa3f9a2a4f8d eval/runs_saved/phase3_evaluation/TC1_v3_verified_first_try
cp -r eval/traces/fa3f9a2a4f8d eval/runs_saved/phase3_evaluation/TC1_v3_verified_first_try/traces
```

This is now the demo plan. 4 scenes, 80 seconds total — perfect for a Phase 3 video.

Now the rendering branch

Two important things from your `claude_client.py`:

1. The text-mode call is just `call_claude` (not `call_claude_text`). I had the wrong name. I'll fix that in the agent.
2. Your existing `animation_coder.py` prompt says "use MathTex with LaTeX" — but you don't have LaTeX installed (you went with the lighter Manim install). MathTex will crash at render time. **I need to rewrite the prompt to forbid MathTex entirely.**

Here's the patch sequence. Apply in this order:

Patch 1: `src/prompts/animation_coder.py` — rewrite, no LaTeX

Replace the entire file:

```
"""System prompt for Agent 4a: Animation Coder (Phase 3)."""

ANIMATION_CODER_SYSTEM_PROMPT = """You are the Animation Coder in Aurua. You turn a verified scene
plan into Manim Community Edition code that renders as a short, clear educational animation.

You will receive ONE scene at a time with: scene_id, claim, visual_goal, duration_sec.
```

OUTPUT REQUIREMENTS

1. Return ONLY the complete Python file content. No markdown fences. No prose. No explanations.
2. Begin with: ``from manim import *``
3. Define exactly one class named ``Scene{NN}`` where NN is the zero-padded scene_id (e.g. ``Scene01``, ``Scene02``).
4. The class must inherit from ``Scene`` and implement ``def construct(self):``.
5. The scene's runtime must approximate duration_sec within ± 3 seconds.

HARD CONSTRAINTS – VIOLATING ANY OF THESE WILL CRASH THE RENDERER

- **NO ``MathTex``.** NO ``Tex``. ** LaTeX is not available. Use ``Text(...)`` for everything, including mathematical notation. Write equations as plain Unicode strings: ``Text("∂C/∂w")``, ``Text("z = w·a + b")``, ``Text("−∇C")``.
- **NO 3D scenes.** Use ``Scene`` only, never ``ThreeDScene``.
- **NO external assets.** No image files, no SVG imports, no audio. Everything must be drawn with Manim primitives.
- **NO ``manim_ml`` or third-party plugins.** Only `stdlib` + ``from manim import *``.

ALLOWED PRIMITIVES

- Shapes: ``Circle``, ``Square``, ``Rectangle``, ``Dot``, ``Line``, ``Arrow``, ``DoubleArrow``, ``CurvedArrow``, ``Ellipse``
- Text: ``Text`` (with optional ``font_size``, ``color``, ``weight=BOLD``)
- Grouping: ``VGroup``, ``Group``
- Layout: ``.next_to(...)``, ``.shift(...)``, ``.move_to(...)``, ``.to_edge(...)``, ``.arrange(...)``
- Animations: ``Create``, ``Write``, ``FadeIn``, ``FadeOut``, ``Transform``, ``ReplacementTransform``, ``Indicate``, ``GrowArrow``, ``DrawBorderThenFill``
- Pauses: ``self.wait(seconds)``

STYLE GUIDE

- Use a dark background (Manim default). Use bright colors for emphasis: ``YELLOW``, ``BLUE``, ``GREEN``, ``RED``, ``WHITE``.
- Add ``self.wait(0.5)`` to ``self.wait(1.5)`` between visual beats so narration has room to breathe.
- Keep object counts modest. A scene with 4-8 named objects is plenty.
- Position elements with ``.next_to()`` and ``.shift()`` rather than absolute coordinates where possible. The frame is roughly 14 wide × 8 tall in Manim units.

EXAMPLE OUTPUT FORMAT (this is the literal shape – your scenes will be richer)

```
from manim import *

class Scene01(Scene):
    def construct(self):
        title = Text("Gradient Vector", font_size=36, color=YELLOW)
        title.to_edge(UP)
        self.play(Write(title))
        self.wait(1)

        vec = VGroup(
            Text("∂C/∂w1", font_size=28),
```

```

        Text(" $\partial C / \partial w_2$ ", font_size=28),
        Text(" $\partial C / \partial b_1$ ", font_size=28),
    ).arrange(DOWN, buff=0.4)
    self.play(FadeIn(vec))
    self.wait(2)

    self.play(FadeOut(vec), FadeOut(title))

```

The class will be rendered with:

```
manim -ql -o scene_NN <file>.py SceneNN
```

Now generate the complete file for the requested scene.

```
"""
```

Patch 2: `src/agents/animation_coder.py` — create new file

Create this file (your `agents/` dir doesn't have it yet — only the prompt stub exists):

```

"""Agent 4a: Animation Coder.

Takes a verified scene plan and generates Manim code for each scene.
One Claude call per scene, sequential for trace clarity.

Writes generated code to outputs/runs/<run_id>/scenes/scene_NN.py
where the Renderer (deterministic tool) picks them up.
"""

from __future__ import annotations

import re
from pathlib import Path

from ..claude_client import call_claude
from ..prompts.animation_coder import ANIMATION_CODER_SYSTEM_PROMPT
from ..state import RunState, RunStatus, Scene
from .. import config

def run_animation_coder(state: RunState) -> RunState:
    """Generate Manim code for each scene in the verified plan."""
    if state.scene_plan is None or state.status != RunStatus.VERIFIED:
        # Don't generate code for unverified plans – that's the whole point
        # of having the verifier in the first place.
        return state

    scenes_dir = Path(config.RUNS_DIR) / state.run_id / "scenes"
    scenes_dir.mkdir(parents=True, exist_ok=True)

    trace_dir = Path(config.TRACES_DIR) / state.run_id

    for scene in state.scene_plan.scenes:
        code = _generate_scene_code(scene, state, trace_dir)
        state.animation_code[scene.scene_id] = code

```

```

        # Write to disk for the Renderer
        scene_file = scenes_dir / f"scene_{scene.scene_id:02d}.py"
        scene_file.write_text(code, encoding="utf-8")

    state.status = RunStatus.CODE_COMPLETE
    return state

def _generate_scene_code(scene: Scene, state: RunState, trace_dir: Path) -> str:
    """Single Claude call for one scene."""
    user_prompt = (
        f"Generate Manim code for this scene.\n\n"
        f"scene_id: {scene.scene_id}\n"
        f"claim: {scene.claim}\n"
        f"visual_goal: {scene.visual_goal}\n"
        f"duration_sec: {scene.duration_sec}\n\n"
        f"Class name: Scene{scene.scene_id:02d}\n"
        f"Output file will be: scene_{scene.scene_id:02d}.py\n\n"
        f"Return ONLY the complete Python file content. No markdown fences."
    )

    raw = call_claude(
        system_prompt=ANIMATION_CODER_SYSTEM_PROMPT,
        user_prompt=user_prompt,
        model=config.CODER_MODEL,
        temperature=config.CODER_TEMPERATURE,
        cost_tracker=state.cost,
        trace_label=f"04a_animation_coder_scene_{scene.scene_id:02d}",
        trace_dir=trace_dir,
    )

    return _strip_code_fences(raw)

_FENCE_RE = re.compile(r"```(?:python|py)?\s*\n(?:.*?)\n```s*$", re.DOTALL)

def _strip_code_fences(text: str) -> str:
    """Remove ```python ... ``` wrappers if Claude added them despite instructions."""
    text = text.strip()
    match = _FENCE_RE.match(text)
    if match:
        return match.group(1).strip()
    return text

```

Patch 3: `src/tools/renderer.py` — create new file

This is the deterministic tool that compiles scene code into video. No LLM, just subprocess + ffmpeg.

```

"""Deterministic Renderer (not an agent).

```

Takes generated Manim scene code and compiles it into video files using the manim CLI. Optionally muxes audio in via ffmpeg.

This module is deliberately simple: subprocess calls, no LLM, no retries beyond what manim itself provides. If a scene fails to render, we log the error and skip it rather than failing the whole run.

```
"""
from __future__ import annotations

import subprocess
from pathlib import Path
from typing import Optional

from ..state import RunState, RunStatus
from .. import config

def render_all_scenes(state: RunState, quality: str = "low") -> RunState:
    """Render every scene in state.animation_code to an MP4.

    Args:
        state: RunState with animation_code populated.
        quality: 'low' (480p15, fast), 'medium' (720p30), or 'high' (1080p60).

    Sets state.status = RENDERED on success, FAILED on no-scenes-rendered.
    Stores rendered video paths in state.scene_video_paths.
    """
    if not state.animation_code:
        state.error = "render_all_scenes called but state.animation_code is empty"
        state.status = RunStatus.FAILED
        return state

    run_dir = Path(config.RUNS_DIR) / state.run_id
    scenes_dir = run_dir / "scenes"
    videos_dir = run_dir / "videos"
    videos_dir.mkdir(parents=True, exist_ok=True)

    quality_flag = {"low": "-ql", "medium": "-qm", "high": "-qh"}[quality]

    rendered: dict[int, str] = {}
    failed: list[int] = []

    for scene_id in sorted(state.animation_code.keys()):
        scene_file = scenes_dir / f"scene_{scene_id:02d}.py"
        class_name = f"Scene{scene_id:02d}"

        if not scene_file.exists():
            failed.append(scene_id)
            continue

        # Run manim with the scene file. Output goes into media/videos/...
        # We use --media_dir to keep everything inside the run folder.
        media_dir = run_dir / "manim_media"
```

```

cmd = [
    "manim", quality_flag,
    "--media_dir", str(media_dir),
    "--output_file", f"scene_{scene_id:02d}",
    str(scene_file),
    class_name,
]

print(f"\n[renderer] rendering scene {scene_id} → {' '.join(cmd)}")
result = subprocess.run(cmd, capture_output=True, text=True)

if result.returncode != 0:
    print(f"[renderer] scene {scene_id} FAILED:")
    print(result.stderr[-2000:]) # tail of stderr to keep output manageable
    failed.append(scene_id)
    # Save stderr for debugging
    (run_dir / f"render_error_scene_{scene_id:02d}.log").write_text(
        f"STDOUT:\n{result.stdout}\n\nSTDERR:\n{result.stderr}",
        encoding="utf-8",
    )
    continue

# Find the produced MP4 (manim puts it in a quality-named subdir)
produced = _find_rendered_mp4(media_dir, f"scene_{scene_id:02d}")
if produced is None:
    print(f"[renderer] scene {scene_id} ran but no MP4 was found")
    failed.append(scene_id)
    continue

# Copy to canonical location for easy access
target = videos_dir / f"scene_{scene_id:02d}.mp4"
target.write_bytes(produced.read_bytes())
rendered[scene_id] = str(target)
print(f"[renderer] scene {scene_id} → {target}")

state.scene_video_paths = rendered

if not rendered:
    state.error = f"All {len(state.animation_code)} scenes failed to render"
    state.status = RunStatus.FAILED
else:
    state.status = RunStatus.RENDERED
    if failed:
        state.error = f"Partial render: {len(failed)} of {len(state.animation_code)} scenes
failed: {failed}"

    return state

def _find_rendered_mp4(media_dir: Path, basename: str) -> Optional[Path]:
    """Locate the MP4 that manim produced. Manim's path layout varies by version."""
    for candidate in media_dir.rglob(f"{basename}.mp4"):
        return candidate

```

```

return None

def concatenate_videos(state: RunState) -> Optional[Path]:
    """Concatenate all rendered scene videos into a single output.

    Returns the path to the concatenated MP4, or None on failure.
    """
    if not getattr(state, "scene_video_paths", None):
        return None

    run_dir = Path(config.RUNS_DIR) / state.run_id
    concat_list = run_dir / "concat_list.txt"
    output = run_dir / "final_silent.mp4"

    # ffmpeg concat demuxer needs a file listing
    sorted_paths = [state.scene_video_paths[sid] for sid in sorted(state.scene_video_paths)]
    concat_list.write_text(
        "\n".join(f"file '{p}'" for p in sorted_paths),
        encoding="utf-8",
    )

    cmd = [
        "ffmpeg", "-y",
        "-f", "concat",
        "-safe", "0",
        "-i", str(concat_list),
        "-c", "copy",
        str(output),
    ]
    print(f"\n[renderer] concatenating {len(sorted_paths)} scenes → {output}")
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        print(f"[renderer] concat failed:\n{result.stderr[-2000:]}")
        return None

    return output

```

Patch 4: `src/state.py` — add `scene_video_paths` field

Find the `RunState` class. Right after the line `animation_code: dict[int, str] = Field(default_factory=dict)`, add:

```
scene_video_paths: dict[int, str] = Field(default_factory=dict) # scene_id -> mp4 path
```

Patch 5: `src/run.py` — add rendering branch

Find the `try:` block in `run_pipeline`. After the verifier loop ends and before the `except`, add this block. The exact spot is right after the comment `# CLARIFICATION_REQUIRED – handled above, unreachable here`:


```
# --- Phase 3: Animation Coder + Renderer ---
# Only run rendering if verification passed cleanly.
if state.status == RunStatus.VERIFIED:
    from .agents.animation_coder import run_animation_coder
    from .tools.renderer import render_all_scenes, concatenate_videos

    state = run_animation_coder(state)

    if state.status == RunStatus.CODE_COMPLETE:
        state = render_all_scenes(state, quality="low")
        if state.status == RunStatus.RENDERED:
            final = concatenate_videos(state)
            if final:
                print(f"\n[run] final silent video: {final}")
```

Patch 6: Verify Manim works *before* burning a real run

Before we trigger the rendering pipeline, make a dead-simple Manim file and render it. If your Manim install is broken, we want to find out now, not after we've burned API tokens generating four scene files.

Create `scripts/test_manim.py`:

```
"""Smoke test: confirm Manim renders a trivial scene."""
from manim import *

class HelloWorld(Scene):
    def construct(self):
        title = Text("Aurua renderer is alive", font_size=48, color=YELLOW)
        circle = Circle(radius=1.5, color=BLUE).next_to(title, DOWN, buff=0.8)

        self.play(Write(title))
        self.play(Create(circle))
        self.wait(1)
        self.play(Indicate(circle))
        self.wait(1)
        self.play(FadeOut(title), FadeOut(circle))
```

Then run:

```
manim -ql scripts/test_manim.py HelloWorld
```

This should produce `media/videos/test_manim/480p15/HelloWorld.mp4`. If you get an MP4, your install works and we can move on. If you get an error, paste it.

Order of operations

1. **Save TC1 v0.3 run** (the `cp` commands at the top)
2. **Apply patches 1-5** (prompts, agent, renderer, state field, orchestrator)

3. **Run the Manim smoke test** (patch 6)

4. **If smoke test passes:** re-run TC1 — this time it'll generate code and render. Use the same command as before, since v0.3 retrieval is now baked in.

```
python -m src.run \
  --question "What does the gradient of the cost function tell us about which weights to change?" \
  --source data/transcripts/3b1b_ch3.txt
```

Expected: ~8 API calls (query expansion + intent + planner + verifier + 4 scenes), ~\$0.50 (Opus is more expensive than Sonnet for the coder), 2-3 minutes wall-clock for the LLM calls + several minutes for Manim rendering.

Paste back two things:

- The smoke test output (success or error)
- If smoke test passes, the full pipeline output

If anything errors, send the full traceback. Don't try to fix it yourself — let me see the raw output.

Go.

4/28/2026, 10:55:09 PM



File: state.json

File:

File: 00_query_expansion.txt

File: 01_intent.txt

File: 02_planner_initial.txt

File: 02_planner_retry_0.txt

File: 03_verifier_attempt_0.txt

File: 03_verifier_attempt_1.txt

Here's what I got. The manim test video was created successfully! But we got an error on the first try for the question. The scenes folder in aurua/eval/traces/97fb7e6b6768/ is empty

4/28/2026, 11:01:52 PM

Two big wins, one specific bug to fix. Let me unpack:

Wins:

1. Manim works. The test rendered cleanly through 6 animations.
2. The verifier loop is *still* doing real work — TC1 v0.3 hit the verifier on attempt 0, got a precise rejection ("conflates sensitivity with influence/importance, a subtle but meaningful distinction"), the planner revised the claim correctly, and attempt 1 passed. Different evidence than the previous TC1 run we archived. Save this one too — it's another clean demo of the loop catching a real overreach.

The bug:

```
BadRequestError: temperature is deprecated for this model.
```

This hits when the Animation Coder makes its first API call. Your config has `CODER_MODEL = "claude-opus-4-7"` with `CODER_TEMPERATURE = 0.3`. The error is telling us this specific model no longer accepts a `temperature` parameter at all. It's a recent API change — Opus 4.7 only accepts the default temperature.

The fix is to make `temperature` conditional in `claude_client.py`. We just don't send the parameter when it's not supported. Here's the patch:

Patch: `src/claude_client.py`

Find `call_claude` and replace the `client.messages.create(...)` block. The current code is:

```
response = client.messages.create(
    model=model,
    max_tokens=max_tokens,
    temperature=temperature,
    system=system_prompt,
```

```

    messages=[{"role": "user", "content": user_prompt}],
)

```

Replace with:

```

# Build kwargs conditionally – some recent models (e.g., claude-opus-4-7)
# have deprecated the `temperature` parameter and reject it outright.
create_kwargs = {
    "model": model,
    "max_tokens": max_tokens,
    "system": system_prompt,
    "messages": [{"role": "user", "content": user_prompt}],
}
if not _model_rejects_temperature(model):
    create_kwargs["temperature"] = temperature

response = client.messages.create(**create_kwargs)

```

Then add this helper at module level (anywhere outside the function definitions, e.g. right above `_extract_json`):

```

# Models that have deprecated the `temperature` parameter. Add to this list
# as the API surface changes. Calling with temperature on these models
# returns 400 invalid_request_error.
_MODELS_WITHOUT_TEMPERATURE = frozenset({
    "claude-opus-4-7",
})

def _model_rejects_temperature(model: str) -> bool:
    return model in _MODELS_WITHOUT_TEMPERATURE

```

Save the new TC1 run before re-running

Don't lose this one — it's actually richer evidence than the first verified run because it shows the verifier catching a *subtler* mistake (sensitivity vs. influence). Different demo angle.

```

cp -r outputs/runs/97fb7e6b6768
eval/runs_saved/phase3_evaluation/TC1_v3_verified_retry1_subtle_correction
cp -r eval/traces/97fb7e6b6768
eval/runs_saved/phase3_evaluation/TC1_v3_verified_retry1_subtle_correction/traces

```

Now you have two phase 3 verified TC1 runs. The first-try one (`fa3f9a2a4f8d`) is the cleaner demo. The retry-1 one (`97fb7e6b6768`) is the better failure-loop demo. Both are useful.

Re-run

After applying the patch:

```
python -m src.run \
  --question "What does the gradient of the cost function tell us about which weights to change?" \
  --source data/transcripts/3b1b_ch3.txt
```

Expected this time:

- Verifier passes (could be first try or retry, both fine for our purposes)
- Then 4 calls to `claude-opus-4-7` for the Animation Coder, one per scene
- Then 4 Manim subprocess calls for rendering
- Then ffmpeg concat to a single silent video

Total wall-clock estimate: ~3-5 minutes, mostly from Opus generation (slower than Sonnet) + Manim render time. Cost ~\$0.50-1.00.

Watch the console output as it goes. If the Animation Coder succeeds, you'll see traces appear in

`eval/traces/<run_id>/04a_animation_coder_scene_*.txt`. If a Manim render fails, you'll see `[renderer] scene N FAILED` followed by error tail — paste that and we debug.

One thing to be honest about: this is the first time we're calling Opus on a real task in the project. If Opus's generated Manim code has bugs (wrong API usage, syntax errors, infinite animations), the renderer will catch it but we'll need to iterate the prompt. Don't be surprised if scene 1 works and scene 3 has a bug — that's normal first-pass.

Apply the patch, re-run, paste back what happens. While you do that I'll start drafting the Narration Writer (Tier 2) so it's queued up.

4/28/2026, 11:02:28 PM



File: state.json

File:

File: concat_list.txt

File: scene_01.py

File: scene_02.py

File: scene_03.py

File: scene_04.py

File: 00_query_expansion.txt

File: 01_intent.txt

File: 02_planner_initial.txt

File: 02_planner_retry_0.txt

File: 03_verifier_attempt_0.txt

File: 03_verifier_attempt_1.txt

File: 04a_animation_coder_scene_01.txt

File: 04a_animation_coder_scene_02.txt

File: 04a_animation_coder_scene_03.txt

File: 04a_animation_coder_scene_04.txt

Here's what we got. It outputted a video of 1 minute 22 seconds and honestly the visuals do not look bad. It's pretty interpretable.

4/28/2026, 11:11:04 PM

End-to-end pipeline working. Take the win.

Status: **rendered**, 4 verified scenes. 4 Manim renders. ffmpeg concat to a single MP4. 1 minute 22 seconds of actual educational video, generated by the agent pipeline from a student question, all citations grounded, all on Aurua's first attempt at this. That is a real artifact.

A few quick observations before we move on:

- **The verifier loop did its job again.** Attempt 0 rejected scene 1 because the planner overreached on "vector where each component corresponds to a specific weight" — the cited span talks about *sensitivity*, not vector structure. Planner re-anchored on a different span, attempt 1 passed all 4. Different rejection reason than the last run. You now have three documented verifier-catches-overreach examples (Phase 2 TC1, Phase 3 retry-1 first run, this run). Good evidence pattern.
- **Cost: \$0.50 for the whole pipeline.** 10 calls total, 105 seconds of API time. Cheap.
- **Save this run too.** This is now your canonical Phase 3 demo:

```
cp -r outputs/runs/170daccbae47 eval/runs_saved/phase3_evaluation/TC1_v3_FULL_RENDERED
cp -r eval/traces/170daccbae47 eval/runs_saved/phase3_evaluation/TC1_v3_FULL_RENDERED/traces
```

Now we keep moving. You said tier-2 (audio) is on the table. Let me lay out the rest of the night.

Remaining Phase 3 work

#	Task	Est. time	Status
1	Tier 2: Narration Writer (Agent 4b)	30 min	Up next
2	Tier 2: TTS with OpenAI tts-1 per scene	30 min	After narration
3	Tier 3: ffmpeg mux audio + video → final MP4 with sound	45-60 min	Stretch
4	Per-agent RunState read access (grader's deduction)	20 min	Should-do
5	Re-run TC6 (single call, methodology fix)	5 min	Should-do
6	Orchestration justification section (grader's explicit ask)	30 min	Required for report
7	Update evaluation_results.csv + failure_log.md + version_notes.md with v0.3 data	60 min	Required
8	Final report with v0.3 results, before/after narrative, rendered artifact	90 min	Required
9	Architecture diagram update (now that rendering branch is real)	20 min	Required
10	README + AI_USAGE update	30 min	Required

That's about 5-6 more hours if we go full tier-3. Tier-2 alone shaves 45-60 min off.

Now: Tier 2 — Narration Writer

The Narration Writer takes the verified plan and produces a script with per-scene timing markers. It's much simpler than the Animation Coder because the output is text, not code.

Need three things from you to write precise patches. Run this and paste:

```
cat src/agents/narration_writer.py
echo "===== "
cat src/prompts/narration.py
```

Both exist as Phase 2 stubs in your tree. I want to see how much is already drafted so I either fill in the gaps or replace cleanly. While you grab those, the design plan:

Output format from Narration Writer — per-scene narration, structured so TTS can be done one chunk per scene (matches the per-scene rendering pattern, makes audio/video mux clean):

```
class SceneNarration(BaseModel):
    scene_id: int
    text: str # the narration spoken during this scene
    target_duration_sec: int # matches the scene's planned duration

class NarrationScript(BaseModel):
    scenes: list[SceneNarration]
```

Constraint baked into the prompt: the narrator's text for each scene must take roughly `duration_sec` seconds to read aloud. We tell it to target ~145 words per minute (slow educational pace) and produce roughly `duration_sec * 145 / 60` words per scene. We also instruct it to ground claims in the same source spans the planner used, so the narration doesn't drift from what the verifier approved.

Add to RunState: `narration_script: Optional[NarrationScript] = None` and `scene_audio_paths: dict[int, str] = Field(default_factory=dict)`.

While I wait for those two files, let me also pre-draft the prompt so we save a round trip:

```
Create File: Draft the Narration Writer prompt
```

And the agent skeleton:

```
Create File: Draft the Narration Writer agent
```

Both drafted. They reference `NarrationScript` which doesn't exist in your `state.py` yet — I'll send that patch in the next message once you paste the existing files. There's also the `RunStatus.NARRATION_COMPLETE` enum value that I can see is already in your `state.py` from the Phase 2 scaffold, so that's fine.

Paste those two files (`narration_writer.py` and `narration.py`) and I'll send the full patch set:

1. `src/state.py` — add `NarrationScript`, `SceneNarration`, and the `narration_script` field
2. `src/prompts/narration.py` — replace with the new prompt
3. `src/agents/narration_writer.py` — replace with the new agent
4. `src/run.py` — add the narration call after `run_animation_coder`
5. **A test command** to run narration in isolation against the existing run (so we don't burn an Opus run just to test narration text)

Go.

4/28/2026, 11:12:15 PM



File: Jorge Urias Phase 2 Feedback.pdf

File:

Great! Before going into the audio, do we need to do any more runs for the full video pipeline or testing in accordance with the feedback we got and what the project requirements are? Let's take a step back and look at what we have so far. I've provided the feedback from phase 2 along with the full project requirements (we're looking more so at the phase 3 and final project deliverables).

4/28/2026, 11:23:22 PM

Smart move to pause and audit before sprinting further. Let me walk through what you have and what you genuinely still need.

What you have right now

v0.3 evaluation runs (this session):

- 1. TC2 v0.3 — `c3061f8c2527` — exhausted, but with measurable retrieval improvement (4/4 spans now retrieved that weren't before; deeper finding about source granularity)
- 2. TC1 v0.3 first try — `fa3f9a2a4f8d` — verified first attempt, 4 scenes
- 3. TC1 v0.3 retry-1 — `97fb7e6b6768` — verified after 1 retry (subtle correction), no rendering due to temperature bug
- 4. TC1 v0.3 FULL RENDERED — `170daccbae47` — verified after 1 retry, 4 scenes generated, 4 Manim renders, 1m22s concatenated silent video

Phase 2 archive (carries forward):

- TC1 v0.2, TC2 v0.2 (exhausted), TC3, TC4, TC5 v0.2 (exhausted), TC6 baseline

That's **9 distinct evaluation runs** across two architecture versions, with documented failures and a documented fix.

Mapping against Phase 3 rubric requirements

Let me go category by category through the rubric and the grader's three asks:

Category	Status	Evidence	Gap
Final artifact (20pt)	Done	<code>final_silent.mp4</code> from run <code>170daccbae47</code>	Could add audio for polish, not required
Evidence of agentic behavior (15pt)	Strong	Verifier-rejection-then-revision visible in TC1 v0.3 + TC1 v0.2 traces	None
Evaluation quality (15pt)	Almost	9 runs across 6 distinct test cases	TC6 only ran once (grader flagged); preference test never run
Failure analysis & iteration (15pt)	Strong	Phase 2 documented 3 failures; v0.3 partially closed F1/F2; deeper finding emerged	Need to write up the v0.3 results in <code>failure_log.md</code> and <code>version_notes.md</code>
Governance / responsible behavior (10pt)	Partial	Verifier loop, retry cap, cost tracker, vague-input refusal all live	Disclaimer scene + per-scene citation overlay in rendered output not implemented (grader flagged)
5-min video (10pt)	Not made	—	Thursday morning work
Documentation / portfolio (15pt)	Stale	Has all required files	All need v0.3 updates

Grader's three explicit asks:

1. **✅ Wire Agents 4a/4b + produce rendered video** — Agent 4a wired, video rendered. 4b not wired but the grader's stated minimum was "even if TTS and ffmpeg muxing are not complete." You're already past the minimum bar.
2. **⚠️ Apply v0.3 retrieval fixes and report honestly** — Done for TC2 and TC1. **TC5 was never re-run with v0.3.** That's a gap. The grader specifically named TC2 *and* TC5.
3. **❌ Name and justify orchestration approach** — Not yet written. Needs a section in the Phase 3 report.

Honest assessment of what's missing

In priority order:

Required for Phase 3 (will lose points if not done):

1. **Re-run TC5 with v0.3** — 1 minute API time. Grader explicitly named it. Skipping it is a visible gap.
2. **Re-run TC6 a second time** — methodology fix the grader noted. 30 seconds.
3. **Update** `eval/evaluation_results.csv` with all v0.3 rows
4. **Update** `eval/failure_log.md` with the v0.3 retrieval-fix results (partial closure of F1/F2, plus the new finding about source granularity)
5. **Update** `eval/version_notes.md` with v0.3 retrieval config and outcomes
6. **Final report PDF** — problem, architecture, implementation, evaluation, results, failure analysis, governance, lessons learned. The grader's full ask.
7. **Architecture diagram update** — needs the rendering branch (Animation Coder + Narration Writer + Renderer) since it's now implemented
8. **README + AI_USAGE.md** updates for Phase 3
9. **Screenshot index + 4-8 screenshots** — terminal runs, rendered video frames, trace files, evaluation tables. This is the package-level deliverable the rubric explicitly requires.
10. **Submission packet PDF** — the one-page cover doc Canvas wants
11. **Orchestration justification section** — grader's third ask

Nice-to-have but not required by minimum bar:

- Tier 2 (Narration Writer + TTS audio)
- Tier 3 (ffmpeg mux for audio + video)
- Per-agent RunState read access (the grader noted but said "specifying per-agent read access in Phase 3 would close that" — meaning showing how it would work in your final report could substitute for full implementation if time-constrained)

Phase 3 requires (separate from rubric):

- Individual contribution reflection
- Reflection on improvements and next steps

My honest recommendation

Audio is the wrong priority right now. Here's why:

You already have a working rendered artifact. The grader's stated minimum was "one rendered scene." You have four scenes plus a 1m22s concat. Going from silent-rendered to audio-rendered is a tier of polish, not a required deliverable. The 5-minute video on

Thursday can have your voice narrating over the silent Manim — that's normal for educational explainers and reads as a deliberate choice.

What's actually missing is the documentation layer. The Phase 3 rubric devotes **15 points to documentation/portfolio quality** and **15 points to failure analysis/iteration** — those are 30 of the 100 Phase 3 points, and right now they're stale relative to what your system can actually do.

Better path tonight:

```
Tool: Generating ask_user_input_v0...
```

My strong recommendation: **option 2 (hybrid)**.

Reason — the missing TC5/TC6 runs are a visible gap in your evidence package. The grader is the same Jorge Urias who explicitly asked for them in writing. Walking in with TC5 v0.3 missing is a self-inflicted wound on the "Failure analysis and iteration" 15-pointer. Same for TC6. These are 5 minutes of API time combined.

The doc updates take longer but are also where the points are. Final report alone is a 60-90 minute task.

The reason for "if time permits" on audio: even a partial Tier-2 (just narration script generated, no actual TTS or muxing) would be a nice show on Thursday because you can read the script aloud yourself during the live demo. That's genuinely cheap — one extra Claude call to add to the pipeline.

Tap your pick. If you go with hybrid, the order I'd suggest is:

1. Re-run TC5 with v0.3 (~1 min) — same command pattern as TC2
2. Re-run TC6 a second time (~30 sec)
3. Save those runs alongside the others in `eval/runs_saved/phase3_evaluation/`
4. Update `evaluation_results.csv` + `failure_log.md` + `version_notes.md` with all v0.3 data
5. Write the final report (the big one)
6. Update architecture diagram to include rendering branch
7. Capture screenshots
8. Update README + AI_USAGE
9. Write submission packet
10. **Then** audio if time allows

That's roughly 4-5 hours of focused work. Aggressive but doable. Tap your pick and we move.

4/28/2026, 11:24:20 PM

Q: Given the gaps I just listed, which path tonight? A: Hybrid — doc gaps + missing runs first, audio if time permits at end

4/28/2026, 11:26:58 PM

Good. Let's move.

Step 1 — Re-run TC5 with v0.3

The FAISS index is currently built from Ch3 (last run was TC1). TC5 is on Ch4. So we need to clear and rebuild.

```
rm -rf data/index/

python -m src.run \
  --question "Why does the chain rule give us a sum over paths when a neuron connects to multiple neurons?" \
  --source data/transcripts/3b1b_ch4.txt
```

Watch for two things:

1. Whether the verifier passes (vs. v0.2 which exhausted)
2. Whether retrieval surfaces line 115-119 (the multi-path answer in Ch4)

This will run the *full pipeline* including Manim rendering. That's actually fine — even if TC5 verifies, the rendering step will burn ~\$0.50 of Opus calls and a few minutes of Manim time. If you want to skip rendering for TC5 (since we don't need the video, just the verification result), you can temporarily disable the rendering branch. Up to you:

```
Tool: Generating ask_user_input_v0...
```

My recommendation: **option 1 — run full pipeline.**

Reasons:

- \$0.50 is rounding error at this stage
- A second rendered video gives you a backup demo if TC1's video has anything weird, and lets you show range in the Thursday presentation
- Most importantly: if TC5 verifies on v0.3, that's the cleanest possible "before / after" evidence for the rubric. v0.2 exhausted, v0.3 produces a working video. That's the whole iteration narrative in one example.
- We don't have to handle "what if TC5 verifies but we only have the plan, not the video" — full pipeline removes the asymmetry between TC1 and TC5 in your evidence

Just run the command above as-is.

While TC5 runs, here's what I need next

Once TC5 finishes, you'll paste me back the console output. Then in the same shot:

```
# TC6 second run — same question as TC2, single-prompt baseline
python scripts/run_tc6_baseline.py \
  --question "Why is the derivative of the cost with respect to a weight proportional to the
```

```
activation feeding into it?" \  
--source data/transcripts/3b1b_ch4.txt
```

This is the methodology fix the grader called out. Single API call, ~30 sec, ~\$0.04. We just need a second data point so the comparison isn't N=1.

And after that

Save them:

```
# Replace the run_id with whatever TC5 actually produces  
cp -r outputs/runs/<TC5_RUN_ID> eval/runs_saved/phase3_evaluation/TC5_v3  
cp -r eval/traces/<TC5_RUN_ID> eval/runs_saved/phase3_evaluation/TC5_v3/traces  
  
# Same for TC6 second run  
cp -r outputs/runs/<TC6_RUN_ID> eval/runs_saved/phase3_evaluation/TC6_baseline_run2
```

Then we have the full v0.3 evidence package and can move to documentation.

What I'm doing in parallel

While you run TC5 + TC6, I'm going to start drafting the documentation deliverables in the sandbox so they're ready for review the moment your runs finish:

1. `eval/evaluation_results.csv` — full table with all 9 runs
2. `eval/version_notes.md` — v0.3 section appended
3. `eval/failure_log.md` — F1/F2 status update + the new finding about source granularity
4. Skeleton of the final report — sections, headers, what each section needs

I'll send those as drafts you can then drop in and tweak rather than write from scratch.

Run TC5 now. Paste output. Then run TC6. Paste output. We move from there.

4/28/2026, 11:27:24 PM

Q: Should TC5 v0.3 run with rendering or without? A: Run full pipeline — burn the \$0.50, get a second rendered video as bonus

4/28/2026, 11:32:20 PM

Good. Run TC5 with the full pipeline:

```
rm -rf data/index/

python -m src.run \
  --question "Why does the chain rule give us a sum over paths when a neuron connects to multiple neurons?" \
  --source data/transcripts/3b1b_ch4.txt
```

Three things to watch the console output for:

1. **Retrieved spans** — does any chunk include the line ~115-119 territory ("the neuron influences the cost function through multiple different paths... you have to add those up")? In v0.2 the L115-119 span was rank #6+ and got filtered out by top-5. With v0.3's top-10 + chunk size 40 + query expansion, it should appear in the retrieved set.
2. **Final status** — `verified` or `verification_exhausted`. Either is informative. If verified → the v0.3 fix closed the gap on TC5. If exhausted → we have the same "improved retrieval, deeper finding" story we got from TC2 v0.3.
3. **Rendering completion** — should produce 3-4 scene MP4s and a final concatenated silent video. Estimate ~\$0.50-0.80 total cost, ~3-4 minutes wall-clock.

Paste back the full console output when it finishes.

In the meantime, I'm building the doc skeletons

Two things in parallel — you wait for the runs, I draft. Specifically:

Bash Tool: Set up doc directory in sandbox

Create File: Skeleton of evaluation_results.csv with all known runs

Create File: Updated VERSION_NOTES.md with v0.3 section

Create File: Updated failure_log.md with v0.3 results and the new finding

Three drafts ready in the sandbox. They have placeholder rows for TC5 v0.3 and TC6 run-2 — once you paste the run results, I'll fill those in and ship the final files.

What's queued for after TC5 + TC6

Once those two runs finish:

1. Fill in the `PENDING` rows in `evaluation_results.csv` and `version_notes.md`
2. Ship those three files for you to drop into `eval/`
3. Start drafting:

- The final report (the big 90-min deliverable)
- Updated architecture diagram (now with rendering branch)
- Orchestration justification section (the grader's third ask)

4. Then screenshots, README/AI_USAGE updates, submission packet

Now run TC5 with that command above. Paste the full console output when it finishes.

4/28/2026, 11:35:07 PM



File: state.json

File:

File: 00_query_expansion.txt

File: 01_intent.txt

File: 02_planner_initial.txt

File: 02_planner_retry_0.txt

File: 02_planner_retry_1.txt

File: 03_verifier_attempt_0.txt

File: 03_verifier_attempt_1.txt

File: 03_verifier_attempt_2.txt

Here's what we got

4/28/2026, 11:40:52 PM

Read carefully. This is exactly the F4 finding repeating itself on TC5 — and the verifier surfaced the *real* root cause in plain English on the final attempt.

What happened on TC5 v0.3

Status: `verification_exhausted`, 8 calls, \$0.10, 68 seconds. 3 retries.

Retrieval IS better. In v0.2, the multi-path span (L115-119) was rank 6+ and got filtered. In v0.3, retrieval surfaced spans `5d0b154bb4` (L108-111) and `a7895247a2` (L82-85) — the chain-rule territory the question is about. **But L115-119 itself is still not in the top 10.** Same pattern as TC2: improved coverage, deeper finding emerges.

The verifier's final-attempt rejection nails it:

"the sentence is cut off and does not actually contain the full claim. Specifically, it does not show or state the summed multi-neuron chain-rule equation."

That's not a verifier being too strict. That's the verifier correctly noting that **the chunks are truncated mid-sentence**, so even when the right region of text is retrieved, the chunk boundaries cut off the answer. The 40-word chunk size we tightened to in v0.3 actually made this *worse* than the 80-word v0.2 — smaller chunks have a higher chance of slicing through a key sentence.

This is now a **second concrete finding** for F4 (or arguably F5):

- **F4 (TC2):** Retrieval improved, but the source itself is informal — formal claims can't be grounded.
- **F5 (TC5):** Retrieval improved, but smaller chunks introduced sentence-truncation artifacts that strip the key payload from the cited span.

Both are real, both are honest, and both are *exactly* the kind of "what changed after testing" narrative the rubric rewards. You now have:

	TC2 v0.2	TC2 v0.3	TC5 v0.2	TC5 v0.3
Status	exhausted	exhausted	exhausted	exhausted
Retrieval coverage	Wrong-topic spans	Right-topic spans	Wrong-topic spans	Right-topic spans
Verifier reason	Wrong derivative	Informal source	Wrong derivative	Sentence truncation
Resolution	Phase 3 retry	Documented as F4	Phase 3 retry	Documented as F5

That table is your iteration narrative. Honest, specific, traced.

Save it

```
cp -r outputs/runs/d922193d24df eval/runs_saved/phase3_evaluation/TC5_v3_partial_close
cp -r eval/traces/d922193d24df eval/runs_saved/phase3_evaluation/TC5_v3_partial_close/traces
```

Now TC6 second run

Last evaluation gap. 30 seconds, \$0.04. The methodology fix.

```
python scripts/run_tc6_baseline.py \
  --question "Why is the derivative of the cost with respect to a weight proportional to the
  activation feeding into it?" \
  --source data/transcripts/3b1b_ch4.txt
```

Paste the output. Then we have all 11 runs in the evidence package and we can finalize the docs.

What's still being prepared in parallel

The drafts I started are now stale — they had F4 only. After TC6 finishes, I'll send you:

1. **Updated** `evaluation_results.csv` with TC5 v0.3 + TC6 run-2 rows filled in
2. **Updated** `version_notes.md` with the v0.3 final results and the chunk-truncation finding
3. **Updated** `failure_log.md` with both F4 (TC2) and F5 (TC5) entries — the chunk-truncation finding is genuinely interesting and worth its own entry
4. Then we move to the **final report** which is the big writeup the rubric wants

Run TC6, paste output, we move.

4/28/2026, 11:41:16 PM



File: state.json

Output:

```
python scripts/run_tc6_baseline.py \ --question "Why is the derivative of the cost with respect to a weight proportional to the activation feeding into it?"
--source data/transcripts/3b1b_ch4.txt Run ID: tc6_2c72dedd Approach: single-prompt baseline (no retrieval, no verifier) Cost:
2906 in, 816 out in 1 call (11.97s) Plan: 4 scenes, 76s total [Scene 1] The cost depends on the weight through a chain:  $w(L) \rightarrow z(L) \rightarrow a(L) \rightarrow C$ . source quote: "the weight, previous action and the bias all together are used to compute z, which in turn lets us c..."
source location: L031-L033 visual: Show three small labeled boxes in a row: ' $z^L = w^L \cdot a^{(L-1)} + b^L$ ', then ' $a^L = \sigma(z^L)$ ', then ' $C = \dots$ ' duration: 18s [Scene 2] By the chain rule,  $dC/dw(L) = (dz/dw) \cdot (da/dz) \cdot (dC/da)$ , breaking the sensitivity into three multiplicative pieces. source quote: "This right here is the chain rule, where multiplying together these three ratios gives us the sensit..." source location: L052-L053 visual: Display the chain rule equation  $dC/dw^L = (dz^L/dw^L) \cdot (da^L/dz^L) \cdot (dC/da^L)$  as three colored fra... duration: 20s [Scene 3] The derivative of  $z(L)$  with respect to  $w(L)$  equals  $a^{(L-1)}$ , the activation feeding into the weight. source quote: "And the derivative of  $z(L)$  with respect to  $w(L)$ ... In this case comes out to be  $a^{(L-1)}$ ..." source location: L063 visual: Show the formula  $z^L = w^L \cdot a^{(L-1)} + b^L$ . Then animate taking the derivative with respect to  $w^L$ , ... duration: 18s [Scene 4] This means the influence of a weight on the cost is proportional to the strength of the previous neuron's activation — n source quote: "the amount that the small nudge to the weight influenced the last layer depends on how strong the pr..." source location: L066-L068 visual: Show two neurons connected by a weighted edge. Animate the left neuron glowing brighter (large  $a^{(L-1)}$ ... duration: 20s Full result saved to: /Users/papijorge/Downloads/College/FourthYear/AgenticTech/Project/Phase2/Agentic Technologies Course Project/aurua/outputs/runs/tc6_2c72dedd/state.json
```

4/28/2026, 11:45:32 PM

TC6 run-2 reproduces the baseline finding cleanly. Let me make this concrete:

TC6 reproducibility check

	Run 1 (tc6_ddbcd2db)	Run 2 (tc6_2c72dedd)
Cost	\$0.043 / 17.0s	\$0.040 / 12.0s
Scenes	4	4
Quote citations	All 4 verbatim from transcript	All 4 verbatim from transcript
Lines cited	L31-33, L52-53, L63, L66-68	L31-33, L52-53, L63, L66-68 (identical)
Hallucinated content	0	0
Claim drift	None	None

The baseline is stable. Same model, same temp 0.0, same prompt, same source — produces functionally identical plans 12 days apart. The F3 finding holds: the single-prompt baseline reliably outperforms the multi-agent pipeline on this corpus size, on the primary anchor question.

This is the methodology fix the grader specifically asked for ("TC6 ran once rather than the two runs per case called for in \$6.3"). Done.

Save it

```
cp -r outputs/runs/tc6_2c72dedd eval/runs_saved/phase3_evaluation/TC6_baseline_run2
```

Now we have the complete v0.3 evidence package

11 distinct evaluation runs:

#	Test	Version	Status	Notes
1	TC1	v0.2	verified (retry 1)	Phase 2 baseline
2	TC2	v0.2	exhausted	F1 retrieval gap
3	TC3	v0.2	clarification	Out-of-scope refusal
4	TC4	v0.2	clarification	Vague input refusal
5	TC5	v0.2	exhausted	F2 retrieval gap
6	TC6	v0.2	n/a (baseline)	F3 finding
7	TC2	v0.3	exhausted	F4: retrieval better, source informal
8	TC1	v0.3 first try	verified	Cleaner than v0.2
9	TC1	v0.3 (subtle correction)	verified (retry 1)	Different overreach
10	TC1 v0.3 FULL RENDERED	v0.3	rendered	Canonical demo
11	TC5	v0.3	exhausted	F5: chunk truncation finding
12	TC6	v0.3 (run 2)	n/a (baseline)	F3 reproduced

That's 5 documented failures and 1 rendered demo. The rubric wants ≥ 2 failures. We have F1, F2, F3, F4, F5 — five distinct failure modes traced to specific causes with specific evidence.

Now the documentation pass

I'm going to ship you finalized versions of the three eval files. While I prepare those, you have one quick parallel task to set up:

```
# Make sure all v0.3 runs are archived
ls eval/runs_saved/phase3_evaluation/
```

Should show:

- `TC1_v3_FULL_RENDERED/`
- `TC1_v3_verified_first_try/`
- `TC1_v3_verified_retry1_subtle_correction/` (if you saved this one earlier — if not, save now)
- `TC2_v3/` (if not, save now: `cp -r outputs/runs/c3061f8c2527 eval/runs_saved/phase3_evaluation/TC2_v3/` + traces)
- `TC5_v3_partial_close/`
- `TC6_baseline_run2/`

Tell me what's there. If anything's missing, I'll tell you the cp commands.

Now let me ship the three updated eval files. The drafts in my sandbox are stale (they had F4 only), so I'm rewriting them with all the new data.

```
Create File: Updated evaluation_results.csv with all 12 runs
```

Bash Tool: Remove stale draft

Create File: Final evaluation_results.csv with all 12 runs

Bash Tool: Remove stale drafts

Create File: Final version_notes.md with all 12 runs and both v0.3 findings

Create File: Final failure_log.md with all 5 documented failures

Now let me copy these to outputs so you can grab them.

Bash Tool: Copy finalized eval files to outputs

Present File: /mnt/user-data/outputs/eval/evaluation_results.csv, /mnt/user-data/outputs/eval/failure_log.md, /mnt/user-data/outputs/eval/version_notes.md

Three eval files ready. Drop them in:

```
mv ~/Downloads/evaluation_results.csv eval/  
mv ~/Downloads/failure_log.md eval/FAILURE_LOG.md          # match your existing capitalization  
mv ~/Downloads/version_notes.md eval/VERSION_NOTES.md
```

(Note your existing tree had `FAILURE_LOG.md` and `VERSION_NOTES.md` — uppercase. Match that.)

Where we stand

Done tonight:

- ✓ v0.3 retrieval fixes wired and tested
- ✓ TC2 v0.3, TC1 v0.3 (×3), TC5 v0.3, TC6 run-2 — all 6 v0.3 evaluations executed
- ✓ Animation Coder (Agent 4a) wired
- ✓ Renderer wired with Manim + ffmpeg
- ✓ End-to-end pipeline working: question → rendered 1m22s video
- ✓ 5 documented failure cases (F1-F5), 2 partially-closed via fix-then-retest, 1 reproduced for stability
- ✓ Three eval files updated with all v0.3 data

Remaining for Phase 3 submission:

1. Final report PDF (the big one — 90 min)
2. Architecture diagram update (rendering branch added)
3. Orchestration justification section (grader's third ask) — folds into the report
4. README update for v0.3 state
5. AI_USAGE.md update for tonight's session
6. Screenshot index + 4-8 screenshots
7. Submission packet PDF

For Thursday presentation:

- 5-minute video walkthrough

Next up: the Final Report. That's the biggest single deliverable left and it folds in the orchestration justification. I'll draft section by section in the sandbox while you confirm the eval files look right.

One quick question before I start writing:

Tool: Generating ask_user_input_v0...

My recommendation: **Markdown + I render to PDF in the sandbox.**

Reasons:

- Markdown source means you can tweak any line without re-running anything
- I can render with `wkhtmltopdf` (already proven to work — that's how I rendered your Phase 2 architecture diagram and submission packet)
- Cleaner output than docx (which has formatting inconsistencies in the docx skill)
- The course wants a PDF for submission anyway

Tap your pick. Once I know the format, I'll start writing the report. While I write, you can:

1. Drop the three eval files into your project
2. Save the missing v0.3 runs if any are missing from `eval/runs_saved/phase3_evaluation/`
3. Take a 10-minute break — you've been at this for a while

File: `/mnt/user-data/outputs/eval/evaluation_results.csv`

File: `/mnt/user-data/outputs/eval/failure_log.md`

File: `/mnt/user-data/outputs/eval/version_notes.md`

4/28/2026, 11:50:09 PM

 Q: Final report format — what works best for you? A: Final report as Markdown + I render to PDF in the sandbox

4/28/2026, 11:54:23 PM

Good. Writing the final report now. Given how long this is, I'll write it as a single large file in the sandbox, then render to PDF, then present both.

A few orientation choices I'm making before I start writing — flagging them so you can push back early:

Voice and length. The Phase 2 grader rewarded honest, specific writing over polished marketing. I'll write in the same register: concrete claims with file/line evidence pointers, no hand-waving, room to admit limits. Target length: 12-18 pages. The rubric rewards "well organized, easy to navigate" — not maximum word count.

Structure. Following the rubric's exact required sections: Problem, architecture, implementation, evaluation, results, failure analysis, governance, lessons. The orchestration-justification asked for by the grader becomes a subsection inside the architecture chapter.

Stance on the failures. F1, F2, F3 are Phase 2 carry-forwards. F4 and F5 are Phase 3 findings. I'll frame the v0.3 work honestly: "we ran the planned fixes; here's what improved, here's what got worse, here's what got more interesting." That's the iteration narrative the rubric rewards. We do NOT spin the partial closures as full successes.

One thing I'm explicitly NOT going to do: write a triumphant "Aurua works end-to-end" framing. The honest framing is "Aurua produces a verified, grounded, rendered video on TC1, and we documented exactly where it doesn't work and why." That's a stronger submission than overclaim.

Let me write.

```
Create File: Final report — full draft
```

Report drafted. Now render to PDF.

```
Bash Tool: Check tools available for PDF rendering
```

Both available. Use pandoc → wkhtmltopdf which will give a cleaner styled output than markdown direct.

```
Bash Tool: Render report to PDF
```

Some Unicode character is breaking wkhtmltopdf encoding. Let me try with weasyprint (cleaner Unicode support) or fall back to a different approach.

```
Bash Tool: Check for weasyprint
```

```
Bash Tool: Find which non-ASCII characters might be the problem
```

That's just em dashes and apostrophes — not the issue. Let me look for the actual U+FFFD replacement character (which is what `\65533` is).

```
Bash Tool: Find U+FFFD if present
```

```
Bash Tool: List all non-ASCII chars
```

All real, valid Unicode. The issue is wkhtmltopdf's font handling. Let me try a different approach — go through HTML with explicit UTF-8 meta tag, embedded styling, then to PDF:

Bash Tool: Generate styled HTML

Bash Tool: Render HTML to PDF

Bash Tool: Check PDF size

123KB PDF. Let me verify it actually rendered correctly by checking page count and grabbing a preview:

Bash Tool: Check page count

Bash Tool: Render page 1 as image to verify

View: Verify page 1 rendered correctly

Two issues:

1. **The H1 title shows** `Aurua ❖❖❖ Final Report` — em dash didn't render in the title font. Same for the body H1. Body em dashes elsewhere look fine. This is a font fallback issue with wkhtmltopdf for the heading typeface.
2. **List items in body content are running together as inline text** ("- Diagnose..." showing as part of the same paragraph). The bullet-point lists aren't formatting properly.

Let me fix both. For the em dash, I'll use `-` (en dash) in headings or just hyphenate. For the list rendering, need to check the markdown source.

Bash Tool: Check list formatting in source

Lists ARE properly formatted in markdown. The issue is the CSS `max-width: 720px; margin: 0 auto;` is causing the body's lists to be rendered without `<ul>` styling, which in pandoc usually works but may be conflicting here. Let me inspect the HTML:

Bash Tool: Check if lists rendered as ul in HTML

Bug. Pandoc didn't render those as `<ul><li>` — it kept them as inline text with literal hyphens. Likely cause: the previous line ends with `:` and then the bullets start without a blank line separator. Let me check:

View: Check line context around the broken list

Yep, pandoc requires a blank line between a paragraph and a list. Quick fix: add the blank lines. Same problem may exist throughout — let me batch-fix it.

Bash Tool: Add blank lines before lists where missing

Only 7 bytes added — most of the file's lists were already separated. Now also fix the H1 em dash issue. The cleanest fix is to use a hyphen-minus (`-`) in the H1 only since the heading font fallback can't handle em dash:

Edit: Replace em dash in main H1 with hyphen-space-hyphen

Wait — en dash might also fail with the same heading font. Let me try plain hyphen in headings only and rebuild.

Edit: Use plain ASCII hyphen in title for safety

Also fix the `--metadata title` flag we'll pass to pandoc. Now rebuild:

Bash Tool: Rebuild with fixes

Bash Tool: Render preview pages

View: Verify page 1 fixed

Title fixed. But the list under "The job to be done" is still inline. Let me check — the regex earlier may have missed some cases. Looking at the source content showing in the PDF:

"The job to be done. Given a source and a question: - Diagnose what the student is actually confused about (mechanical math vs. conceptual intuition vs. genuinely vague). - Produce a short animated explainer..."

Let me look at the actual current source:

View: Verify the fix actually wrote correctly

The script didn't catch it because line 30 starts with `**The job to be done.**` which my regex saw as starting with `*`. Let me just manually fix the few problem spots and rebuild.

Edit: Fix list separation for The job to be done

Now let me find any other instances of `**word.**\n-` and fix them at once:

Bash Tool: Better regex to add blank lines before all lists

Only 2 more bytes. Let me also check if there are issues elsewhere. Quickly scan for remaining suspect patterns:

Bash Tool: Find any remaining unseparated lists

Good. Now rebuild:

Bash Tool: Rebuild and render previews

View: Verify page 1 fixed

Page 1 looks clean. Title renders. Lists render. Body em dashes render correctly. The TOC heading "Aurua: Final Report" appearing as a list item before the title is just pandoc's standard behavior (the doc title gets included in the TOC) — minor cosmetic issue but not worth more time.

Let me check a few more pages to make sure everything looks good:

View: Page 3

4/29/2026, 12:00:52 AM